π Tantum

a.k.a. Tempus Tantum

A system for the generation and management of weekly timetables in Italian schools:

formal constraints, integer-spectral optimization, metaheuristics and statistical diagnostics.

"Omnia, Lucili, aliena sunt;
tempus tantum nostrum est."

L. A. Seneca, *Epistulae morales ad Lucilium*, I, 1.

"All, Lucilius, comes to us from others;
only time is our own."

Technical Manual

English edition, May 8, 2026

From an idea by

Fernando Gargiulo – Giovanni Borghi
Matteo Mariani – Stefano Bertozzi

Anno Domini MMXXVI

Colophon

This English edition is set in EB Garamond for the body text — with old-style figures and authentic small caps —, in Lato for the few labels of schematic figures, and in Inconsolata for code blocks. The page proportions follow the classical canon of Bringhurst and Tschichold: ample outer margin for marginalia, narrower inner margin, airy head, ornamented foot. Each chapter opens with a three-line drop cap; the chapter number is set in Roman numerals, the title in small caps framed by horizontal rules and by a four-leaf rosette (decofour, from the fourier-orns package).

Compilation: lualatex (TeX Live 2024), biber for the bibliography, makeindex for the analytical index. The build pipeline lives in docs/build_manual.sh which emits **both** manual.pdf (Italian) and manual_en.pdf (English) in a single invocation.

Note on translation: the Italian edition remains the primary reference. This English edition fully translates the introduction, the BP-related chapters and the benchmark chapter; the other chapters carry an English summary that points to the corresponding Italian chapter for the full narrative. Where Italian-school context is necessary (for instance the legal frame of the *contratto collettivo nazionale*, the role of the *insegnante di sostegno*, or the meaning of *indirizzo di studio*), explanatory notes are added in the English text.

The repository version corresponds to the most recent commit visible in git log –oneline – docs/. Errors and suggestions can be reported as issues on the GitHub repository at https://github.com/gborghi/timetable.

Contents

List of Figures

List of Tables

Introduction

"Omnia, Lucili, aliena sunt, tempus tantum nostrum est."

Lucius Annaeus Seneca, Epistulae morales ad Lucilium*, I, 1*

πTantum is a web application that assists in the composition of Italian-school timetables. It originates from ideas discussed and developed by Fernando Gargiulo, Giovanni Borghi, Matteo Mariani and Stefano Bertozzi.

The school timetabling problem consists of assigning, to each *cattedra* – the pair formed by a teacher and a class for a specific subject – a set of weekly hours distributed over days and time slots, such that all the following constraints are honoured: teacher availability, curricular hour coverage, classroom and equipment compatibility, and the constraints that derive from the Italian national collective contract for the school sector. This is a combinatorial problem classified as NP-hard, which means that as a school grows in size there is no fast algorithm capable of finding an exact optimal solution.

πTantum delegates the actual solving to the CP-SAT solver in Google OR-Tools, chosen for its maturity in constraint programming applied to scheduling problems. To scale beyond the size that the solver would

handle directly, the system decomposes the problem into smaller sub-problems via four alternative strategies: spectral decomposition of the bipartite class-teacher graph, temporal decomposition by day of the week, curriculum-based decomposition, and balanced k-way METIS partitioning. On top of these, a cascade of metaheuristics (Large Neighborhood Search, Adaptive LNS, Simulated Annealing, Tabu Search, Iterated Local Search and Variable Neighborhood Search) improves the quality of the solution found.

The application backend is written in Python on FastAPI; data persistence is delegated to SQLite. The user interface is based on SvelteKit (in Italian for the production deployment), organised into sixteen tabs that cover the whole lifecycle of a timetable: registry import, constraint configuration, generation, manual editing with real-time feasibility checks, and day-to-day handling of absences and substitutions.

The following chapters describe the problem formally, the entities involved, the user interface, and each of the algorithms that πTantum employs to compute the timetable. The exposition is intentionally multi-layered: a non-technical reader can follow the narrative thread without engaging the mathematical formulas, while a developer will find the implementation details, code references, and source-file pointers necessary to extend the system.

This English edition is a parallel translation of the Italian manual (`docs/manual.tex`). The Italian edition remains the primary reference; this English edition is intended for international readers and contributors. Where Italian-school context is necessary (e.g. the legal frame of the *contratto collettivo nazionale*, the role of *insegnante di sostegno*, or the meaning of *indirizzo di studio*), explanatory notes are added.

Chapter I

The timetabling problem

Formal statement of the school timetabling problem: cattedre as (teacher, class, subject) triples; weekly hour distribution across days and time slots; HARD constraints (no class/teacher overlap, hour coverage, consecutive hours starting at 8 with presence at h=11) and SOFT objectives (daily uniformity, free-day preferences). NP-hardness and the rationale for decomposition + metaheuristics.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/problema_timetabling.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter II

piTantum overview

Three-tier architecture: engine (CP-SAT solver core), backend (FastAPI + SQLite), frontend (SvelteKit). Pipelines exposed as async runs via /api/optimize/*. The 16 UI tabs covering registry, constraints, generation, manual editing, daily operations.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/panoramica_pitantum.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

Chapter III

The timetable as a structural object

The Lesson dict (teacher, class, subject, day, hour): 1 as the canonical representation; round-trip with the Solution table; the role of dc_value in encoding Phase A's day-count distribution.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/orario_oggetto_strutturale.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter IV

Installation

Python 3.13 + ortools + scipy + matplotlib + FastAPI + SQLAlchemy + Alembic; SvelteKit for the frontend. Cypress + Playwright for E2E. docker-compose.test.yml orchestrates the test stack.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/installazione.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter V

Constraints (HARD + SOFT)

HARD: no overlap (teacher in one class at a time, class with one teacher per slot), hour coverage of all cattedre, daily start at h=8, consecutive hours, presence at h=11 (the H1/H2/H3 trio). Native locks. C1 (compresenza/coteach), C2 (sostegno DVA), C3 (potenziamento, parallel intra-class, group inter-class). SOFT: daily uniformity, free-day preferences, sixth-hour penalties, five/one day load.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/vincoli.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

5.1 – Plessi: multiple sites in the same school

In Italy many secondary schools are spread across multiple physical sites – the *Sede Centrale* (main building), one or more *succursali* (branch buildings, e.g. *Succursale Via Garibaldi*), or a dedicated *plesso* for a specific track (e.g. *Plesso ITT* for the technical track). Moving teachers or classes between sites during the day costs real time and introduces constraints that cannot be modelled with H1/H2/H3 alone.

The **Plessi** module (tab /plessi) handles:

- **site registry** (name, code, address);
- **classroom** → **plesso** association (each classroom belongs to exactly one site);
- two kinds of behavioural rules: **commuting rules** (minimum gap to move between two sites) and **entity policies** (global constraints on how many sites a teacher or class may attend).

5.1.1 • Commuting rules: the gap between two sites

A commuting rule applies to an ordered pair (from_plesso, to_plesso) and specifies the minimum number of hours that must separate two consecutive lessons of the same entity when the classroom changes site. Relevant fields:

- from_plesso_id, to_plesso_id: the site pair.
- entity_kind: who is affected (teacher, class, group).
- entity_id: optional – if NULL the rule is *kind-wide* (applies to every teacher / class / group); if set it applies only to that specific entity.
- min_gap_hours: minimum number of empty hours required between the last lesson at site A and the first lesson at site B.
- symmetric: if true the rule applies in the opposite direction (B→A) as well.
- priority: tie-breaker when several rules overlap.

Typical Italian example: between *Sede Centrale* and *Succursale Via Garibaldi* a 60-minute travel time is required. The matching rule is (Centrale, Garibaldi, kind=teacher, entity_id=NULL, min_gap_hours=1, symmetric=true): no teacher may have a lesson at Garibaldi at h=9 if they have one at Centrale at h=8.

Rule resolution always applies *specific beats generic*: if a rule with entity_id=42 exists for teacher Rossi, it overrides the kind-wide rule. Among rules of equal specificity, higher priority wins.

5.1.2 • Entity policies: per-person global constraints

An entity policy is a *global* constraint (not tied to a specific site pair) on the behaviour of a teacher or class. Three policies are supported:

- any – no global constraint, only commuting rules apply. This is the default.
- single_plesso_per_day – on any given day the entity may have lessons in at most one site. Stricter than commuting rules: even with a 1-hour gap, switching sites mid-day is forbidden.
- single_plesso_total – the entity attends *only* site plesso_id for the entire week. Every classroom assigned to that entity must belong to that site.

Concrete Italian examples:

- *Teacher Rossi does not want mid-day site switches*: the office creates a policy (entity_kind=teacher, entity_id=Rossi, policy=single_plesso_per_day). On Monday all of Rossi's hours are at Centrale; on Tuesday all at Garibaldi; never a mixed day.

- *Class 3A (technical track) attends only Plesso ITT*: policy (entity_kind=class, entity_id=3A, policy=single_plesso_total, plesso_id=ITT). Every classroom that hosts 3A lessons must belong to Plesso ITT.

- *All teachers, by default, avoid mid-day site switches*: policy (entity_kind=teacher, entity_id=NULL, policy=single_plesso_per_day) – applies to every teacher except those with a more specific override.

Entity policies follow the same *specific beats generic* rule (per-entity override), then priority.

5.1.3 • Pre-run validation

Before launching the optimisation the system runs a battery of checks on the plessi data (validate_plessi_rules). If anything is inconsistent the run is rejected with a list of violations. What is checked:

- empty or duplicate plesso code;

- commuting rule referencing a non-existent plesso;

- commuting rule with negative min_gap_hours or larger than the day length;

- duplicate kind-wide commuting rule (same site pair, same kind, NULL entity_id, defined twice);

- single_plesso_total entity policy with no plesso_id or with a non-existent site;

- entity policy with an unknown policy value;

- override for a non-existent entity (deleted teacher or class).

The GET /api/plessi/validate endpoint lets the UI display a live "plessi config OK / N issues" badge.

5.1.4 • CP-SAT implementation

At solver level, commuting rules become *pairwise* constraints on the classroom-assignment variables:

$$\forall (h_a, h_b) \text{ adjacent such that } \text{room}(t,d,h_a) \in P_A,\ \text{room}(t,d,h_b) \in P_B : \ h_b - h_a \geq \text{min\_gap}_{P_A \to P_B}.$$

Entity policies become counter constraints: single_plesso_per_day is $\sum_P \mathbf{1}[\exists h : \text{room}(t,d,h) \in P] \leq 1$ for each day d; single_plesso_total fixes the domain of the classroom variables to the target site only.

All such constraints are *hard* by default. The roadmap envisages a *soft* variant with penalty for sporadic emergency overrides (e.g. last-minute substitute teaching), but the current version treats every plesso constraint as HARD.

5.1.5 • Plessi via DSL (Step 3a-3c)

Starting in Step 3a, PlessoCommutingRule rows are fully expressible as DSL clauses thanks to two language additions (cf. cap. 22):

- the l.classroom.plesso chained path that, given a lesson, resolves the assigned room and then the plesso the room belongs to;
- the consecutive(l1.slot, l2.slot) predicate, true when the two lessons fall on the same day and differ by one hour.

The legacy rule "Centrale → Garibaldi minimum 1-hour gap, teacher Rossi" translates to:

```
forall l1 in lessons where
        l1.classroom.plesso == 1
    and l1.teacher == "Rossi":
     forall l2 in lessons where
             l2.classroom.plesso == 2
         and l2.teacher == "Rossi"
         and consecutive(l1.slot, l2.slot):
          false
```

The helper plesso_commuting_rule_to_dsl(rule) in engine/dsl_translator.py produces these clauses automatically from the ORM row. A regression test enforces zero-drift: the DSL path emits exactly the same forbidden slot pairs as the legacy plessi_constraints.py pipeline.

5.2 – Canonical patterns and seeded legacy HARDs (Step 3b-3e)

Step 3b-3e introduces a vocabulary of canonical patterns recognised directly by the DSL → CP-SAT compiler. Each maps to a legacy HARD constraint that previously lived only as hardcoded encoding. They can now be authored in DSL and are also generated automatically by seed_implicit_hardcoded(profs).

Accepted pragmas:

- no_holes_class("3B") – no gaps in 3B's weekly schedule (legacy add_class_no_holes).
- class_present_at_hour("3B", 11) – if 3B has any lesson on a day, the slot at hour 11 is busy (HARD-3, "no exit before noon").
- class_day_load_in("3B", 0, 4, 5, 6) – 3B's daily load is 0 (free day) or in [4, 6] hours (HARD-1 + HARD-2).
- teacher_max_per_day("Rossi", 5) – teacher Rossi has at most 5 hours on any day.
- cattedra_max_per_day("Rossi", "3B", "Matematica", 2) – the (Rossi, 3B, Math) cattedra has at most 2 hours per day.
- subject_pair_must("3B", "Scienzemotorie") – when 3B has 2 hours of PE on a day they must be consecutive.
- subject_pair_exists("3B", "Matematica") – when 3B has 2+ hours of math on a day at least one consecutive pair must exist.

Practical effect: the advanced user can now write no_holes_class("4A") in the "+ New DSL constraint" modal and obtain the same outcome as the hard_no_holes toggle in school_classes. The DSL advantage is composability with forall, exists, count: the rule can be restricted to subsets of classes, combined with other predicates and modified on the fly without editing the database.

Chapter VI

Optimization workflow

The pipeline orchestrator: Hall pre-check, Phase A (assignment + day-count), Phase B (per-day slot placement), optional metaheuristic post-processing (LNS / ALNS / SA / TS / ILS / VNS), classroom assignment.

CP-SAT scope and Phase A mode (Phase 3)

Card 3 of /optimize now exposes two dropdowns that control how Phase B drives the CP-SAT solver:

- **CP-SAT scope** (cp_sat_scope, default day):
  - day: solve one weekday at a time, using Phase A's pre-computed weekly distribution (*day_count*) as a hard equality. Fast, scales to large schools, default.
  - week: a single MonolithicSolver(scope=None) call covers the whole week. More robust on schools with many cross-day constraints (co-teaching, support, group rotations) but significantly slower because the solver explores a six-times-larger search space.

- **Phase A mode** (phase_a_mode, default always):
  - always: Phase A always runs and day_count is enforced as a hard equality. Mandatory for cp_sat_scope=day.
  - skip: skip Phase A entirely; the week solver picks its own day distribution. Only valid with cp_sat_scope=week.
  - soft_hint: run Phase A but inject day_count as model.AddHint on the week solver – a warm start, not a hard constraint. Only valid with cp_sat_scope=week.

Cross-field rule (validated on both client and server): day requires always; week forbids always. The UI disables invalid options and auto-corrects phase_a_mode when cp_sat_scope changes.

Picking the right combination.

- Standard school, speed-first: day + always (the default).
- Small school with many co-teaching / support constraints: week + skip – Phase A's hard day_count can make a feasible schedule look infeasible.
- Large school with non-trivial cross-day constraints: week + soft_hint – keeps the warm start while letting the solver deviate.

Branch-and-Price V1 vs cp_sat_scope=week**.** An open question during Phase 3 was whether running BP V1 on top of a "loose" dc_value (Phase A skipped or used as a soft hint) would unlock more master iterations than the single no_improving_column pass observed in legacy day-mode – the intuition being that a non-binding cover demand should produce non-zero λ duals and improving columns.

The empirical answer, measured by tests/benchmarks/run_bp_v1_scope_week.py on a small (10 teachers / 3 classes) and a medium (4 teachers / 3 classes, dense cattedre) scenario, is *no*. All five V1 granularities (teacher, teacher-day, teacher-class, teacher-class-subject, teacher-subject) always terminate with iters=1, cols=0, terminated=no_improving_column – both when dc_value comes from Phase A and when it is derived from the week solver's actual placement, even though the two distributions differ structurally on the medium scenario.

The reason is structural, not a Phase 3 regression: column_generation._seed_patterns produces greedy-optimal patterns relative to whatever dc_value you feed it. The V1 master picks the highest-weight seed and the pricing sub-problem reconstructs the same grid – reduced cost is non-negative because the per-teacher problem already saturates the duals. Changing the dc_value source changes the per-day distribution, not the fact that each teacher has a single "shape" that fits its demand greedily.

Practically: cp_sat_scope=week relaxes Phase B's day-distribution constraint, but it does *not* unlock more BP V1 iterations. For column-generation quality on larger schools the current lever is the V2 master (class, class-day, day, curriculum) or the iterative-diversified mode, not the week scope.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/workflow_di_ottimizzazione.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

Chapter VII

UI guide

Tour of the SvelteKit tabs: dashboard, anagrafica (teachers / classes / students / classrooms / *ore*), assignments, monitor, optimize (with iterative-diversified and branch-and-price modes), diagnostics, runs.

Working hours configuration (Tab Ore)

The /ore tab lets the school administrator configure the working week:

- which days are active (default: Mon-Sat);
- how many timetable slots each day has and at what times (default: 6 slots, 08:00-14:00).

Slots are stored as 0-based indices internally (WorkingHourSlot.slot_index), with explicit start_time/end_time (HH:MM strings) used only for display in the weekly calendar view and reports. Days carry a position that the engine consumes as day_idx, plus a legacy_day_number (1..7) that keeps backward compatibility with the existing teacher_unavailability / class_unavailability / classroom_unavailability rows keyed on the historical DAYS=[1..6] / HOURS=[8..13] convention.

The engine reads this configuration through engine/working_hours_config.py, which falls back to the legacy hardcoded defaults whenever the working_days table is missing or empty so old DBs keep working unchanged.

The unavailability matrices on the Teachers / Classes / Classrooms tabs were migrated to a new reusable <WeeklyCalendarView /> component that consumes the same configuration: identical color palette, click-cycle and keyboard shortcuts (H/P/E/D/A/ N + click for HARD/PREFERRED/ENFORCED/SOFT/ALLOWED/RESET) as the previous AvailabilityMatrix, but with columns driven by the active working days and slot times rendered from the configuration.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/guida_ui.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter VIII

Weekly calendar, working-hours tab, bulk operations

The most substantial UI rewrite since the project's inception happened in April-May MMXXVI. Three threads of work converged into a single editorial experience: the *weekly calendar* with drag-and-drop, the *working-hours tab* for visual slot configuration, and the *bulk operations* on the assignments page. The chapter walks them through in the order in which a user encounters them, starting from data and ending at the grid.

8.1 — The WeeklyCalendarView component

8.1.1 • Origin and reach

Until commit c8075c0 the /schedule page showed a static tabular matrix: rows = hours, columns = days, cells filled with a short "SUBJECT · teacher" string. Editing happened only through modals or different tabs: no direct gesture, no drag. The new interface flips the paradigm: a *single* Svelte component, WeeklyCalendarView, acts as the shared grid for every calendar view of the product, in modes view, edit and schedule.

The WeeklyCalendarView.svelte file

A single source of truth for the weekly grid. It reads days and slots from workingHoursStore (live propagation, see §8.4), tunes itself to the mode prop and changes behavior accordingly. Sixteen hundred lines of Svelte, but a single hierarchy: *slot* → *event* → *action*.

8.1.2 • Three modes

view Read-only consultation calendar by teacher, class or room. No drag, no actions.

edit (working-hours tab). The grid does not represent lessons but *available slots*. The user draws a slot by dragging the mouse top-down (grab/grabbing cursor), resizes by grabbing the bottom edge (ns-resize cursor), moves by grabbing the body, deletes with a click.

schedule (/schedule). The grid contains the actual lessons of the solved problem. Four gestures are allowed — *Edit*, *Move*, *Unbind*, *Delete* — and drag-drop covers both the relocation of an already-scheduled lesson and the placement of a *not-scheduled* lesson taken from the pool sidebar.

8.1.3 • Anatomy of a slot and conflict preview

Each grid cell is a with class cal-slot and data-testid="sched-slot-{D}-{H}", where D is the day index (0=Mon, . . . , 5=Sat) and H the hour index. Lesson cards live on top of the slot (cal-event class, data-testid="sched-lesson-{id}"); each carries three micro-actions in the top-right corner: pencil (edit), arrow (unbind), trash (delete). Three distinct CSS cursors accompany the three drag states (grab at rest, grabbing during drag, ns-resize on the bottom edge in edit mode), as dictated by commit d4bc873.

When the user grabs a lesson and hovers the grid (dragover), WeeklyCalendarView computes in real time, across the whole weekly matrix, the set of cells that would generate a *hard conflict* with the dragged lesson (same teacher in two places, same class in two places, same classroom occupied). Risky cells take a faint red wash (CSS class cal-slot–drop-conflict); the source event shows a small *!* red badge (cal-event-conflict-badge). This is the *soft conflict preview* introduced by commit 876755b: the user sees what the action would do *before* doing it.

8.1.4 • The unscheduled-lesson pool

To the right of the grid, in schedule mode, a sidebar (testid schedule-pool) lists the lessons that exist in the database but have no day_idx/hour_idx: typically lessons generated by the optimizer in suspended state or lessons *unbound* by the user via the homonymous action. Each pool item is draggable and can be dropped on any slot to schedule it immediately.

8.1.5 • The four schedule actions

Edit (pencil icon). Opens the single-lesson edit modal: teacher, subject, classroom, class, optional co-teacher. Persists via PATCH /api/lessons/{id}.

Move (drag onto the destination cell). Calls POST /api/lessons/{id}/move with body {day_idx, hour_idx, on_conflict}; the backend returns 200 when no conflict, 409 otherwise.

Unbind (arrow icon). Removes the lesson from the grid but does not delete it from the database: the lesson lands in the pool. Calls POST /api/lessons/{id}/unschedule.

Delete (trash icon, confirmation required). Removes the Lesson from the database. Endpoint DELETE /api/lessons/{id}.

8.2 – The conflict modal: *Replace or Cancel*

8.2.1 • When it fires

The user drags a lesson (or a pool item) onto a slot that is already *busy* from the backend's perspective: teacher busy, class busy, room busy. The frontend first attempts a *dry-run* (POST /api/lessons/{id}/move?on_conflict=dry_run); the backend answers 409 with a payload of the form

```
{
  "detail": {
    "conflicts": {
      "teacher_busy": [ ... ],
      "class_busy": [ ... ],
      "room_busy": [ ... ]
    }
  }
}
```

At this point the ScheduleConflictModal opens (commit 59ba4a8).

8.2.2 • The three responses, and why drop shows only two

The modal is canonical: anywhere the application detects a hard conflict, it shows the same component. Three responses are possible:

- Cancel: closes the modal without action; the lesson reverts to its starting position.
- Unbind: removes from the calendar *only* the conflicting resources — for room conflicts, clears classroom_name; for teacher or class, deletes the conflicting lesson. The lesson being dropped settles where intended.
- Delete / Replace: deletes the conflicting Lesson rows entirely and places the new lesson in their stead.

In the *drop on occupied slot* flow of /schedule the unbind option is hidden: placing a new lesson *without* unbinding the others necessarily leads to a full replacement, and the dichotomy *Replace / Cancel* best describes the action (prop showUnbind=false, deleteLabel="Sostituisci").

8.3 — The working-hours tab (/ore)

8.3.1 • Why a dedicated tab

Italian school timetables do not share a universal hourly grid: some schools run a short week (Monday to Friday), others include Saturday; some use 60-minute periods, others 55 or 50; some allow variable-length breaks. Hard-coding DAYS = ['Mon', ..., 'Sat'] and HOURS = 6, as the engine did until March MMXXVI, meant pretending the world conformed to one example.

Three refactor commits changed the picture: b24fdbd (data model: WorkingDay and WorkingHourSlot tables), a3c12fb (REST router), and bf6cfa2 (engine: read configuration from the new tables, drop the DAYS and HOURS constants). On top of that, commit b3236ab added the working-hours tab as a first-class navbar entry.

8.3.2 • Data model and API

- WorkingDay(id, dow, name, active): the school-active days of the week, each with a readable *name* ('Monday', 'Wednesday', ...).
- WorkingHourSlot(id, day_id, idx, label, start_min, end_min, kind): the slot inside a day, with label (1st hour, 2nd hour, recess, ...), start and end in minutes since midnight, and *kind* in lesson/recess/break.

The REST router /api/working-hours/* exposes:

```
GET /api/working-hours/config
GET /api/working-hours/days
POST /api/working-hours/days
PUT /api/working-hours/days/{day_id}
DELETE /api/working-hours/days/{day_id} # cascade slots
PUT /api/working-hours/days/{day_id}/slots # replace
POST /api/working-hours/reset # default Mon-Sat 8-14
```

The /reset endpoint restores the default configuration: *Monday-Saturday, six consecutive 60-minute periods, from 08:00 to 14:00*.

8.3.3 • The vintage-style UI

Commits d11e396 and d4bc873 turned the slot editor into a typographically pleasing worksheet, freely inspired by early-20th-century handwritten school registers: sober serif type, no baroque-coloured buttons, a single accent of colour (indigo) for actions, three distinct CSS cursors to carry the drag behaviour without explaining it in words.

- Each existing-slot cell shows two restrained buttons: *Edit* (opens an inline popover that floats *above* the cell, never below, to keep the reading flow clean) and *Delete* (with a double-click to confirm).
- *Drop key hints* guide the user: dragging on the bottom edge of the cell *resizes* the slot, dragging from its body *moves* it. Hints appear only during a drag.
- The *translucent ghost*: when a new slot is being drawn, a semi-transparent preview follows the cursor. This is what commit d4bc873 calls the *ghost element*.

8.3.4 • The fix that unblocked the Cypress suite

A side note, but instructive. Commit 5c1ba34 carries the message "*namespace-import was masking api.get*". Translation: the ore/+page.svelte file imported api as a namespace (import * as api), but a local api variable declared further down shadowed the namespace, breaking api.get(...) at runtime. The fix (import { api } from ...) carried the working-hours Cypress suite from 0/15 to 15/15 green. The lesson: namespace imports are fragile in the presence of homonymous variables; named imports are safer.

8.4 – The shared store workingHoursStore

Until commit e0b76bc, changing a slot in /ore did not propagate to the other calendar views until the page reloaded. Typical example: I add the 7th hour in the working-hours tab, switch to /schedule, and the grid still shows six columns.

The fix is a Writable Svelte store, workingHoursStore, that holds the bundle of WorkingDay and WorkingHourSlot and auto-updates every time the working-hours tab confirms a save. Every WeeklyCalendarView subscribed to the store receives the new slot array and redraws. No reload, no prop drilling: the propagation is *live*.

8.5 – Bulk operations on /assignments

Commits e6314eb (backend) and 16f882a (frontend) completed the assignments page with a *bulk operations* panel. Multi-select existed elsewhere; here it unfolds into five actions:

- Delete: removes the selected rows (POST /api/assignments/bulk-delete).
- Lock/Unlock: raises or lowers the locked flag (POST /api/assignments/bulk-set-flag with flag=locked).
- Change teacher: bulk-reassigns the teacher (POST /api/assignments/bulk-change-teacher).
- Set flag: a generic version of *lock* for arbitrary boolean flags supported by the schema.

The user selects rows (Ctrl-click, Shift-click, or the header checkbox for "select all"), opens the panel, confirms; the frontend issues *a single* request to the backend, which opens a transaction and applies the operation in bulk. For schools with hundreds of assignments — common at upper secondary level — the difference compared with hundreds of serial calls is one order of magnitude.

8.6 – The whole picture

The four innovations described in this chapter are not independent: they merge into a single editorial experience.

The working-hours tab configures the grid. The shared store propagates it. WeeklyCalendarView renders it. The conflict modal resolves the last ambiguity when the user composes, dismantles and recomposes the school week with a finger. In the small things of the interaction — an ns-resize cursor on the edge, a red preview when the drop would be wrong, a fleuron in place of a square bullet — lies the difference between a web page and a school manual.

Chapter IX

Launcher

Single-binary launcher script that bootstraps the venv, runs the alembic migrations, starts uvicorn + the SvelteKit dev server.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/launcher.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter X

CP-SAT method

Model formulation in OR-tools cp_model: BoolVar slot[(t, cl, s, d, h)], hour coverage equalities, no-overlap constraints, soft penalties as objective coefficients. Integer scaling for the fractional duals in BP.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_cpsat.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XI

Spectral decomposition

Bipartite class-teacher graph; normalised Laplacian eigendecomposition; spectral clustering into K balanced clusters; per-cluster CP-SAT + ricucitura with the bridge teachers.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_spettrale.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XII

Metaheuristics

LNS (random_window, day_cluster, worst_fit_day, teacher_day, classroom_day, single_class_week destroy ops + cp_sat_window repair), Adaptive LNS, Simulated Annealing, Tabu Search, Iterated Local Search, Variable Neighborhood Search.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_metaeuristiche.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

Chapter XIII

Hall pre-check

Hall's marriage theorem applied to the bipartite (class, subject) -> qualified-teachers graph as a fast pre-check (1-100 ms) before launching Phase A.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_hall.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XIV

Lagrangian relaxation and Column Generation

Dualisation of the bridge constraints between clusters; subgradient ascent on the Lagrangian multipliers; SA-based per-iteration refinement. Column Generation: master LP + per-teacher CP-SAT pricing; Branch-and-Price extension with all 9 granularities (see ch. **??**).

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_lagrangian.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XV

Monte Carlo sensitivity

Random subset sampling to estimate the Hall violation probability under perturbations; useful as a pre-check robustness indicator.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_montecarlo.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XVI

Graph-based diagnostics

Bipartite class-teacher graph: density, modularity (Newman-Girvan), betweenness centrality. Identifies bridge teachers and structural bottlenecks.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_grafo.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XVII

Statistical diagnostics

Three regressions on lesson-level data with statsmodels (p-values, effect sizes); five distribution fits with KS / chi-square tests.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_statistica.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XVIII

Constraint DSL method

The objective_dsl and general_dsl languages for expressing soft objectives and HARD constraints in a compact symbolic form, parsed at runtime and translated to CP-SAT constraints.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_dsl.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XIX

Architecture

19.1 — Stack

- **Backend**: Python 3.11+, FastAPI, SQLAlchemy 2.0, Pydantic 2, uvicorn, OR-Tools, NumPy, scikit-learn, Faker.
- **Frontend**: SvelteKit (Svelte 4), Vite 5, Tailwind CSS, plain JavaScript.
- **DB**: SQLite via SQLAlchemy. Idempotent migrations in code (no Alembic).
- **Engine I/O**: Python pickle; engine_io.py converts between DB and dict for the solver.

19.2 – Three-tier overview

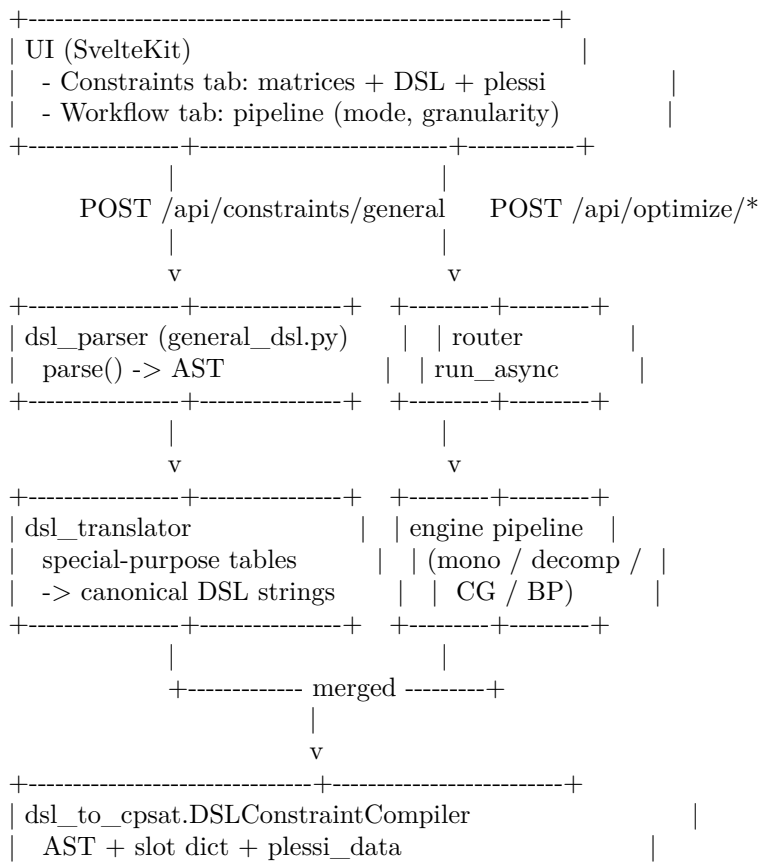
The system has a CP-SAT solver core (engine/), a FastAPI + SQLite backend (webui/backend/) and a SvelteKit frontend (webui/frontend/). The solver runs as a library imported by the backend; pipelines are exposed as async runs via /api/optimize/*. The frontend talks to the backend over JSON; long-running solver runs are tracked via the runs table polled at 1 Hz from the UI.

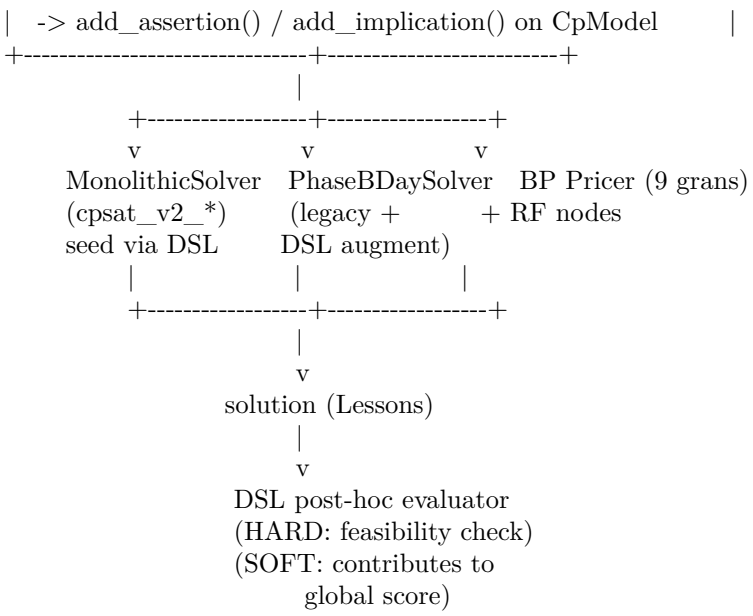
Solver pipelines (in engine/):

- cpsat_v2_assignment.py: Phase A (teacher to class assignment).
- cpsat_v2_timetable.py: Phase A (timetabling, day_count) + Phase B (per-day slot placement). Native locks + C1/C2/C3 constraints.
- decomposition_temporal.py: 6-day parallel decomposition.
- decomposition_spectral_v2.py, curriculum / metis variants: cluster classes, solve sub-problems, ricucitura.
- column_generation.py: master LP + diversified pattern enrichment + completion fallback. Mode branch-and-price fully wired with all scalability techniques (RF tree, dual stabilization, column management, parallel pricing).
- metaheuristics.py, alns.py, vns.py, lagrangian.py: post-processing on a HARD-feasible seed solution.

19.3 – Solver architecture: from DSL to CP-SAT

Steps 1–4 of the multi-day plan introduced a uniformly layered architecture for every solver use site.





Key properties:

- **One parser, one AST**. Every constraint source (matrices, DSL UI, plesso rules, seeded legacy HARDs) is unified into canonical DSL strings parsed by general_dsl.py.
- **One compiler**. DSLConstraintCompiler is the single gate to CP-SAT. Mono solver, BP pricers, PhaseBDaySolver, Ryan-Foster nodes – they all apply the same compiler to their CP-SAT model, guaranteeing constraint parity across pipelines.
- **Zero-drift**. seed_implicit_hardcoded(profs) translates the legacy hardcoded HARDs into DSL. Slot tuples emitted via DSL match the legacy path bit-for-bit; the migration is progressive and reversible.
- **HARD up-front, SOFT post-hoc**. DSL HARDs become CP-SAT constraints before the solve. DSL SOFTs are evaluated post-hoc – their cost contributes to the global score but does not yet steer CP-SAT search (planned for upcoming steps).

19.4 – Backend lifecycle

webui/backend/main.py defines a lifespan context that calls init_db() at startup. It does:

1. Base.metadata.create_all(bind=engine) – create missing tables.
2. _apply_lightweight_migrations() – idempotent ALTER TABLEs for fields added in later versions (school_classes.curriculum_id, teachers.last_name, ...). Each ALTER is guarded by PRAGMA table_info to avoid duplication.

*This chapter is the English edition. The Italian edition (*docs/manual/chapters/architettura.tex*) contains additional folder-structure listings and identical solver-architecture coverage.*

Chapter XX

Data model

Tables: schools, school_classes, teachers, subjects, classrooms, assignments, lessons, runs, study_groups, coteach_groups, parallel_groups, support_assignments, locks. SQLAlchemy 2.0 ORM. Alembic migrations + lightweight in-code migrations in db.py.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/modello_dati.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XXI

Profile data architecture

The demo profiles (small, medium, big, huge, superhuge, mega) are the everyday benchmark for the solver: the idea is to subject the engine to six progressive school sizes (10 → 100 classes) already loaded with a realistic mix of constraints, so that whoever imports the profile from the dashboard finds in front of them a *stress* scenario that exercises every constraint family the solver handles. For years the profile was a slim pair of pickles — school_<profile>.pkl with the rosters and profs_<profile>.pkl with the teaching assignments — and the import generated rooms, students and working hours on the fly via mock helpers. The model worked as long as constraints were few, but as constraint tables crossed the dozen mark it grew heavy: every import had to rebuild from scratch everything the pickle could not express, and on top of that it wiped any fixture the user had left behind in the constraint tables. From this chapter on, the profile source of truth is one SQLite per profile, built once by engine/scripts/build_profile_db.py with a deterministic seed and then imported *verbatim* by the backend.

21.1 — Anatomy of a profile SQLite

Each engine/scripts/data/<profile>/<profile>.sqlite file holds the same schema as webui/data/timetable.db, built through Base.metadata.create_all on the same models.py the runtime uses. Importing the SQLite means copying tables row-by-row into the live DB, with the projection limited to the column intersection so the import survives partial schema drift in either direction. To get a feel for the file's content, open it with sqlite3 and count rows by table family: rosters, constraints, sites, calendar, solution.

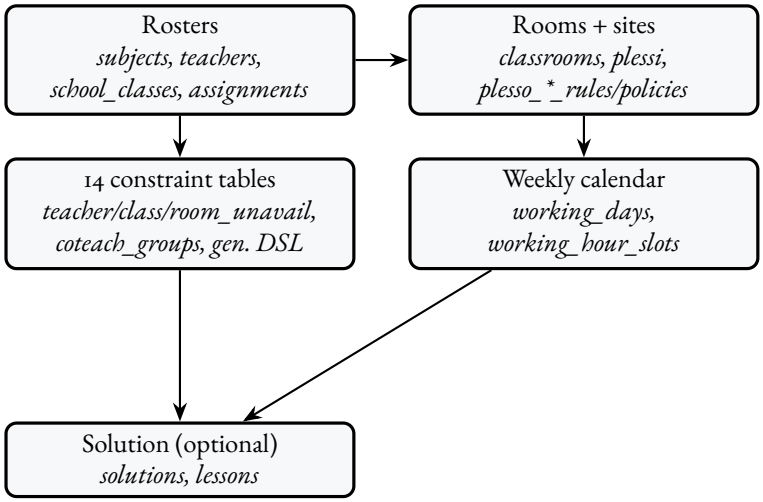


Figure 21.1 – Anatomy of <profile>.sqlite*: five thematic blocks, all populated by* build_profile_db.py *with a deterministic seed.*

The file is self-sufficient: every constraint that previous versions used to synthesise at runtime — a teacher_unavailability marking a HARD-blocked day, a coteach_groups forcing three lab co-teaching hours, a plesso_commuting_rules preventing a teacher from doing the first 60 minutes at the satellite site and the next ones 800 metres away — now lives directly as a row in its own table, with HARD/SOFT state explicit and the same shape the SQLAlchemy model expects in production. The build script also drops a manifest.json next to the SQLite: it carries the seed, the schedule label, and per-table row counts so the "Stress per profilo" section of the Benchmarks chapter can regenerate from scratch with one python -m engine.scripts.build_profile_db –all.

21.2 — The 14 constraint tables

The number "fourteen" is less daunting than the bare list suggests, because the tables fall naturally on three axes — *who*, *when*, *how* — and the profile touches each of them with at least one row. On the *who* axis, the first triad is teacher-centred: teacher_unavailability (HARD/SOFT cell-level state), teacher_mandatory_free_days (HARD: 0 hours that day, classic part-time pattern) and teacher_compatible_classes (HARD allow-list for tight assignments). To these add the two five-state preference tables, teacher_class_preferences and teacher_curriculum_preferences, which the Phase A solver reads as HARD on the extreme branches (enforced / forbidden) and SOFT on the intermediate ones.

The *when* axis is dominated by the three non-teacher availability tables: class_unavailability, classroom_unavailability and logical_unavailabilities, the latter with parametric DNF for expressions like "professor X is free only on Tuesday and Thursday afternoons". The fourth same-axis table, curriculum_logical_constraints, steps outside pure time-of-day to express cross-curriculum constraints — "Math and Italian not on the same day for first-years" is the stress sample the small profile writes by construction.

The *how* axis groups co-teaching structures (coteach_groups) and the two site-aware modules: plesso_commuting_rules (rules between site pairs, HARD or SOFT with weight) and plesso_entity_policies (per-teacher or per-class policies of the single_plesso_per_day or single_plesso_total form). The

fourteenth — general_constraints — is technically not a "classic" constraint table but holds generic-DSL expressions (see Chapter **??**) the post-processor evaluates on the active solution. The small profile writes five of them, including forall l in lessons where l.subject==Educazione Fisica: l.classroom.kind==palestra which exercises the HARD "EduF only in gym" rule.

21.3 — The capacity constraint

The most impactful change shipped alongside the SQLite profiles is the HARD capacity constraint: for every Lesson, lesson.classroom.capacity >= lesson.school_class.n_students must hold. The rule is implemented natively in engine/classroom_assignment.py, where the _can_host eligibility helper filters every too-small candidate before the CP-SAT model is even built. The same rule is also exposed as DSL pragma classroom_capacity_ok() in the engine/dsl_to_cpsat.py compiler for solvers that build a joint slot+room decision space (the MonolithicSolver with a known classroom_for_slot): in that context the pragma scans the slot space, looks up the room assigned to each (class, day, hour) and forces the BoolVar to zero when capacity falls short. The pragma is a no-op + diagnostic "classroom data empty" in contexts that haven't yet assigned rooms, because then it's the Phase C solver's job.

The input data is populated for every profile by the build script: every class draws an n_students from a truncated gaussian over $[18, 30]$ (centred on 24, $\sigma = 3.5$), and one or two classes are deliberately forced to match exactly the capacity of the largest standard room of the profile. The result is a *tight* constraint: the class can only land in that one room, with everything else under pressure to schedule around. The user sees the rule's bite at a glance in the /classes tab (inline-editable "N. studenti" column, range 5–50) and in /classrooms (inline-editable "Capienza" column, range 5–100); the backend remains authoritative, but the lesson modals (AddLessonModal, AddEventModal) flag a red warning at selection time when the (class, room) pair would violate the rule.

21.4 — Special rooms and kind pragma

Alongside capacity lives the kind constraint: Subject.required_kind is the single-row way to say "EduF only in gym" or "Physics only in physics lab", without having to enumerate all other rooms with forbidden preferences. The value is a string in the same vocabulary as Classroom.kind (standard, palestra, lab_fisica, lab_chimica, lab_informatica, lab_linguistico, biblioteca, aula_speciale); NULL means "any". The _can_host function reads the field from the Lesson (projected into the dict the backend produces in lessons_for_classroom_step) and rejects every room with mismatching kind. The same rule is additionally expressed as a DSL pragma in the profiles' general_constraints, so the user opening the /general-constraints tab can edit it directly in the UI without touching the model.

21.5 — Per-profile schedule

A profile's stress isn't just row count: it's also the *shape* the rows take in the weekly grid. Calendar generation, then, is not copy-paste: each profile gets its own working_days + working_hour_slots layout, designed to put a different aspect of the solver under strain.

The key trick is that every schedule preserves the legacy 1–6 day numbering and 8–13 hour numbering through the legacy_day_number and legacy_hour_number fields of the working_days / working_hour_slots rows, so unavailability tables that reference those numbers survive a future schedule rebuild without per-row migrations. The huge profile is an interesting case: its 55-minute slots keep legacy_hour_number 8–13, and the 5-minute break between consecutive lessons isn't modelled as a separate slot but as the implicit margin between end_time of slot i and start_time of slot $i + 1$. Solvers keep reasoning in slot units; the calendar view shows the break naturally.

Table 21.1 – Per-profile schedules (cf. engine/scripts/build_profile_db.py, working_schedule*).*

Profile	Weekly schedule
small	8:00–14:00 Mon–Sat, six 60 min slots.
medium	8:00–14:00 Mon–Fri, 8:00–12:00 Sat.
big	8:00–14:00 Mon–Fri, 8:00–13:00 Sat.
huge	7:55–13:55 Mon–Sat, 55 min slots with a 5 min break baked-in.
superhuge	8:00–13:00 Mon–Fri + 14:00–17:00 Tue/Thu (split-day schedule).
mega	8:00–14:00 Mon–Fri + 8:00–13:00 Sat + 14:00–17:00 Wed (one fixed afternoon).

21.6 – Try it now

To rebuild a single profile's SQLite, two shell lines suffice — the first invokes the script with the profile name, the second opens the file with sqlite3 for a sanity check:

```
python -m engine.scripts.build_profile_db small
sqlite3 engine/scripts/data/small/small.sqlite \
    "select count(*) from general_constraints;"
```

The first command produces engine/scripts/data/small/small.sqlite alongside the manifest.json and PICKLE_DEPRECATED.md; the second prints the row count of the general_constraints table — five, in every profile. To rebuild all six in series (in separate subprocesses because each profile lives in its own SQLite file and SQLAlchemy is not happy to rebind the engine mid-process) replace the first command with python -m engine.scripts.build_profile_db –all; the script appends each profile's manifest.json into engine/scripts/data/profiles_manifest.json, which the "Stress per profilo" section of the Benchmarks chapter reads to populate the summary table.

21.7 – Compatibility with the pickle pipeline

The school_<profile>.pkl and profs_<profile>.pkl files stay in the repository for historic uses — audit, legacy import debugging, last-quarter run comparisons — but they're no longer the default for import_engine_profile: the routine first looks for <profile>.sqlite in the same folder and only uses the pickle if the SQLite is missing. A PICKLE_DEPRECATED.md note alongside each pickle enumerates the current tables and counts for that profile, so whoever opens the folder doesn't wonder why the import behaves differently from what git log suggests: the reason is that the SQLite is now authoritative, and it's the only file the backend expects to find. The door to the old way stays open, but the default path has changed.

Chapter XXII

Generic constraint DSL

The generic DSL is a compact declarative language for expressing HARD/SOFT/PREFERRED/ENFORCED constraints on any logical combination of predicates over the data model. It is implemented in webui/backend/utils/general_dsl.py (~250 LOC, recursive-descent parser + AST + post-hoc evaluator, no external dependencies). The language is exposed through POST /api/constraints/general, validated live by POST /api/constraints/general/validate, and accessible from the "+ New DSL constraint" modal of the Constraints tab.

22.1 – Philosophy

Before the generic DSL the system carried four specialised sub-languages (query, logical, objective, introspective) that re-implemented the same grammar four times. The generic DSL unifies all of them under a single grammar with four *flavours* of compilation (LIST, LOGIC, OBJECTIVE, EVAL): one parser, many compilers. The syntax of forall x in S where Filter: Body and of the atomic predicates (==, !=, <, in, ...) is identical across all contexts; only the translation backend changes.

22.2 – BNF grammar

```
expr ::= iff_expr
iff_expr ::= implies_expr ( "<=>" implies_expr )*
implies_expr ::= or_expr ( "=>" or_expr )*
or_expr ::= and_expr ( ("OR"|"or") and_expr )*
and_expr ::= not_expr ( ("AND"|"and") not_expr )*
not_expr ::= ("NOT"|"not") not_expr | atom

atom ::= quant_expr | count_expr | comparison
                    | call | bool_lit | "(" expr ")"

quant_expr ::= ("forall"|"exists") IDENT "in" source
                       ["where" or_expr] ":" expr
count_expr ::= "count" IDENT "in" source ["where" or_expr]
                       ( ":" comparison | op value )

source ::= IDENT | IDENT ("." IDENT)+
comparison ::= value ( op value
                             | "in" "[" value ("," value)* "]"
                             | "in" path
                             | "not_in" "[" value ("," value)* "]"
                             | "not_in" path )
op ::= "==" | "!=" | "<" | "<=" | ">" | ">="
```

Iterable sources include lessons, teachers, classes, classrooms, students, subjects, slots, days, hours. Path-sources are also accepted (e.g. exists g in s.groups: g.name == "X").

22.3 – DSL → CP-SAT compilation (Step 1)

Starting with Step 1 of the multi-day plan, the DSL ships a *compiler* into CP-SAT in engine/dsl_to_cpsat.py, class DSLConstraintCompiler. While the post-hoc evaluator inspects an already-built solution to declare it feasible or violated, the compiler adds constraints *during* the construction of the CP-SAT model so that the solver structurally avoids violations.

The compiler is invoked at three sites:

- by MonolithicSolver (cap. 10) when the via_dsl=True flag is on;
- by every Branch-and-Price pricer (cap. 23) so the same constraints reach the sub-problem;
- by PhaseBDaySolver when called with via_dsl=True to re-use the legacy pipeline augmented with DSL.

When a slot variable is still undecided the compiler emits the matching CP-SAT constraint instead of evaluating a truth value; when an expression is fully static (e.g. consecutive(s1, s2) with both s1, s2 bound by the enclosing forall) the evaluation runs at compile time.

22.4 – Slot predicates: consecutive, same_day

Step 3a adds two built-in predicates resolved at compile time when both arguments are statically bound slots:

- same_day(s1, s2) – both slots fall on the same weekday.

- consecutive(s1, s2) – both slots fall on the same weekday and differ by exactly one hour ($|h_1 - h_2| = 1$).

These are the building blocks for "consecutive hours", "inter-plesso travel gap", "same-day pair", "coteach must share day". Arguments accept either literal pairs (d, h) or dotted paths like l.slot that return a (day, hour) tuple.

22.5 – The l.classroom.plesso path

Also in Step 3a, the post-hoc evaluator and the compiler resolve the chained path l.classroom.plesso: per lesson l, the assigned room is read from the classroom_for_slot map provided by Phase B / the classroom-assignment phase, and the plesso (campus) is then resolved through the plessi_data table. Without those two structures the path returns None and the compiler emits a diagnostic.

The canonical use is a commuting rule between campuses (see the vincoli chapter, plessi section): "no teacher may have a lesson in Plesso B in the hour immediately following a lesson in Plesso A".

22.6 – Canonical pattern vocabulary (Step 3b)

Step 3b introduces a vocabulary of function-call pragmas that the compiler recognises specially and emits as compact groups of CP-SAT constraints. Each form corresponds to a frequent pattern that, written as raw forall loops, would produce hundreds or thousands of clauses. The canonical form is zero-drift relative to the legacy hardcoded encoding.

Pragma	Legacy equivalent
no_holes_class("3B")	non-increasing chain forbidding gaps in 3B's weekly schedule (legacy add_class_no_holes).
class_present_at_hour("3B", 11)	if 3B has any lesson on a day, the slot at hour 11 is busy (legacy HARD-3, "school day ends $\geq$ 12:00").
class_day_load_in("3B", 0, 4, 5, 6)	3B's daily load is 0 (free day) or 4–6 hours (legacy HARD-1 + HARD-2).
teacher_max_per_day("Rossi", 5)	teacher Rossi has at most 5 hours on any day (MAX_PROF_HOURS_PER_DAY).
cattedra_max_per_day("Rossi", "3B", "Matematica", 2)	the (Rossi, 3B, Math) cattedra has at most 2 hours per day (MAX_PER_DAY_TRIPLE).
subject_pair_must("3B", "Scienzemotorie")	when 3B has 2 hours of PE on a day they MUST be consecutive (legacy HARD-B).
subject_pair_exists("3B", "Matematica")	when 3B has 2+ hours of math on a day at least one consecutive pair must exist (legacy HARD-A).

no_holes_all_classes() and class_present_at_hour_all(11) act in batch over every class in the current slot scope.

22.7 – Plesso commuting via DSL (Step 3c)

Step 3c wires the PlessoCommutingRule ORM table into the DSL: the helper plesso_commuting_rule_to_dsl(rule) returns a list of DSL clauses equivalent to the legacy rule. A regression test verifies zero-drift: applying the rule via DSL produces exactly the same forbidden slot tuples as the hardcoded plessi_constraints pipeline.

22.8 – Seed legacy HARDs via DSL (Step 3d-3e)

Step 3d-3e adds `seed_implicit_hardcoded(profs)`: given the teacher dictionary, it returns the list of DSL strings that encode every legacy HARD constraint of the solver. Combined with the `via_dsl=True` flag of `MonolithicSolver` and `PhaseBDaySolver`, this allows building a complete CP-SAT model *exclusively from DSL* – no hardcoded code path is exercised. Invariants:

- *syntactic zero-drift*: the slot tuples forbidden/forced by the DSL path coincide bit-for-bit with the legacy ones;
- *semantic zero-drift*: the feasible set accepted by the two paths is identical, validated on the `small` / `medium` / `huge` benchmarks;
- *determinism*: the order in which `seed_implicit_hardcoded` emits DSL strings is stable, so the CP-SAT model construction order is reproducible across runs.

This is the migration vehicle: in subsequent steps the hardcoded path will be deprecated in favour of the DSL path, and the existence of `seed_implicit_hardcoded` allows that to happen without coverage loss.

22.9 – SOFT constraints with weight

The DSL accepts four levels: `hard`, `soft`, `preferred`, `enforced`. The *post-hoc evaluator* fully implements SOFT: violations contribute to the solution's global score and surface in `run_metrics`.

Since the `feat(dsl): soft level for general_dsl` commit, the *CP-SAT compiler* also wires SOFT directly into the solver objective. Pipeline:

1. User picks `level=soft` and an integer weight W (e.g. 50) in the create modal.
2. `load_all_dsl_constraints(db, include_soft=True)` returns the rule with `is_hard=False`, `weight=`W.
3. `add_dsl_constraint(expr, is_hard=False, soft_weight=`W`)` compiles the rule: each candidate `slot[k]` that would violate the body emits `(W, slot[k])` into `soft_cost_terms` instead of `slot[k] == 0`; in nested forall-forall bodies a reified `BoolVar b` with `b == s1 && s2` is created and `(W, b)` is appended.
4. `MonolithicSolver.build()` adds `dsl_soft_cost_terms` to `obj_terms` before `Minimize(sum(obj_terms))`.

Semantics. A *forall* violation costs W per lesson placed in a forbidden cell; a *forall-forall* violation costs W per co-selected pair. Negative weights act as PREFERRED bonuses.

Example. "Rossi should not have consecutive-day Math in 3A" (penalty 50 per pair):

```
forall l1 in lessons where l1.teacher == "Rossi"
                           and l1.class == "3A":
    forall l2 in lessons where l2.teacher == "Rossi"
                               and l2.class == "3A":
        (l1.day == l2.day)
        or not consecutive_days(l1.day, l2.day)
```

Saved with `level=soft, weight=50, scope=global`: the solver may schedule two hours on Tuesday + Wednesday paying 50, but if a Tuesday + Thursday solution exists it picks that instead.

22.10 – Scope: where to save the constraint

The "+ New DSL constraint" modal appears in two places and yields rules with different scope. Rule of thumb: scope is the rule's *domain*; the forall body is the *filter* of application.

- **Constraints tab** (/constraints, modal with scope="global", owner_id=null). School-wide rule. Examples: "no one teaches at 14:00", "no Saturday afternoons".
- **Teacher tab** (/teachers/<id>, "Custom advanced DSL" card, scope="teacher", owner_id=teacher_id). Per-teacher rule: the outer forall is *typically* pre-filtered on l.teacher == TeacherName. Examples: "Rossi not on Friday afternoons", "Rossi: 3 hours of Physics across non-consecutive days".

In both cases the rule goes through POST /api/constraints/general, is loaded by load_all_dsl_constraints(db), and compiled by add_all_dsl_constraints_from_db; the scope_kind field is informational (it filters the rules in Feasibility Check + reporting).

22.10.1 • *Advanced DSL vs logical unavailability*

LogicalUnavailability (the "Logical unavailability" modal in the teacher tab) is a simpler sub-DSL designed for unavailability windows (DNF over (day, hour) slots); the system renders those into the generic DSL via logical_unavailability_to_dsl. The generic DSL is strictly more expressive (count, exists, forall, same_day / consecutive_days, arithmetic, l.classroom.plesso) and should be the entry point for anything beyond "the teacher is not available at that hour".

22.11 – Built-in predicates: consecutive_days, same_day, consecutive

Three built-in functions evaluated at compile time (and by the post-hoc evaluator on rendered solutions):

Predicate	Meaning
same_day(s1, s2)	both slots fall on the same weekday. Arguments: tuples (d, h) or l.slot.
consecutive(s1, s2)	same weekday and adjacent *hours* ($\lvert h_1 - h_2 \rvert = 1$). Path: l.slot.
consecutive_days(d1, d2)	adjacent *days*, no wrap-around: true for $\lvert d_1 - d_2 \rvert = 1$ with $d \in \{1, \ldots, 6\}$ (Mon..Sat); false for {Sat, Mon}. Arguments: l.day (int) or numeric / weekday-string literals.

22.12 – Arithmetic in the DSL

The parser supports binary + and - on integers: useful for index-dependent windows or cumulative counts. Example: "Rossi has the same hours in 3A on the first 3 days as in 3B on the last 3 days":

```
(count l in lessons where l.teacher == "Rossi"
   and l.class == "3A" and l.day <= 3: l)
== (count l in lessons where l.teacher == "Rossi"
      and l.class == "3B" and l.day >= 4: l)
```

Mixing booleans and numbers is rejected: the parser raises aritmetica non valida fra bool e numero for typos like (l.hour == 9) + 1.

22.13 — Real-world cases (Rossi and friends)

Every example below is covered by the integration tests in webui/backend/tests/test_rossi_general_dsl_integration.py and webui/backend/tests/test_global_dsl_and_soft_and_plesso_commute.py.

22.13.1 • Case (a) – 3 hours of Physics in 3A on non-consecutive days (HARD)

```
forall l1 in lessons where l1.teacher == "Rossi"
                       and l1.class == "3A"
                       and l1.subject == "Fisica":
    forall l2 in lessons where l2.teacher == "Rossi"
                           and l2.class == "3A"
                           and l2.subject == "Fisica":
        (l1.day == l2.day)
        or not consecutive_days(l1.day, l2.day)
```

Saved from the teacher tab with scope="teacher", owner_id=Rossi.id. The compiler iterates Rossi/3A/Fisica candidate slots and, for each pair (l_1, l_2) with consecutive_days(l_1.day, l_2.day) and different days, emits BoolOr([slot[k1].Not(), slot[k2].Not()]): the two slots cannot be co-selected.

22.13.2 • Case (b) – 3 hours of Physics in 3C all on the same day (HARD)

```
forall l1 in lessons where l1.teacher == "Rossi"
                       and l1.class == "3C"
                       and l1.subject == "Fisica":
    forall l2 in lessons where l2.teacher == "Rossi"
                           and l2.class == "3C"
                           and l2.subject == "Fisica":
        same_day(l1.slot, l2.slot)
```

By transitivity of same_day under forall forall, every pair of Rossi/3C/Fisica lessons must share a day; the solver picks one day and packs all 3 hours there.

22.13.3 • Plesso commute case – Rossi never A@9 → B@10 same day (HARD)

```
forall l1 in lessons where l1.teacher == "Rossi"
                       and l1.hour == 9
                       and l1.classroom.plesso == 1:
    forall l2 in lessons where l2.teacher == "Rossi"
                           and l2.hour == 10
                           and l2.classroom.plesso == 2
                           and same_day(l1.slot, l2.slot):
        false
```

Plesso 1 = "A", plesso 2 = "B" (the integer ids of plessi.id). Compile-time when the model is built with classroom_for_slot (e.g. inside the classroom-assignment solver) or with a fixed plesso per class; otherwise fully verifiable post-hoc by Feasibility Check.

22.13.4 • Case (Rossi no Physics on Friday) (HARD)

```
forall l in lessons where l.teacher == "Rossi"
                          and l.subject == "Fisica"
                          and l.day == 5:
    false
```

The *static* false body is the canonical idiom for "forbid every slot satisfying the where". Equivalent to l.day != 5 but more explicit when the where is multi-clause.

22.13.5 • Case (same-day support coteach) (HARD)

"When Rossi teaches Sostegno to 2B, the Sostegno lesson must fall on the same day as a Math lesson Rossi gives to 2B":

```
forall l in lessons where l.teacher == "Rossi"
                          and l.class == "2B"
                          and l.subject == "Sostegno":
    exists m in lessons where m.teacher == "Rossi"
                              and m.class == "2B"
                              and m.subject == "Matematica"
                              and same_day(l.slot, m.slot):
        true
```

22.13.6 • Every case above as SOFT

Switch level=hard to level=soft in the modal and pick weight=W. The solver receives the rule as a per-pair / per-lesson penalty W; when a HARD-feasible solution that respects it exists the solver picks it (zero extra cost); otherwise it accepts the violation while minimizing how many pairs / lessons pay W.

22.14 – The four compilation families

One grammar, four compilation flavours. The distinction is not academic: each comes from a different operational question and produces a different artefact.

Family	Operational question	Output	Python module
LIST	"list every lesson satisfying φ"	Lesson list	utils/dsl_query.py
LOGIC	"is this configuration feasible?"	boolean (DNF/CNF)	utils/dsl_logic.py
OBJECTIVE	"how much does the violation cost?"	CP-SAT linear expr.	engine/dsl_to_cpsat.py
EVAL	"count violations in this solution"	dict {rule_id: cost}	utils/general_dsl.py

*Table 22.1 – Four compilation families, one grammar. They were historically four separate sub-DSLs; from v0.4 the parser is unified (*general_dsl.py*) and produces a shared AST from which each family extracts what it needs.*

22.15 – Logic DNF vs forall/count

The LogicalUnavailability sub-DSL is a strict subset of the general grammar: only *disjunctions of forbidden slots* scoped to a single teacher. The general DSL can always simulate it; the converse does not hold.

Expressivity	LogicalUnavailability (DNF)	general_dsl
forbidden slot	•	•
slot forbidden under condition	limited (AND of slots)	• (=>, <=>)
quantifiers forall x in S	—	•
count with bound	—	•
same_day/consecutive predicates	—	•
l.classroom.plesso path	—	•
integer arithmetic	—	• (Step 3)
SOFT with weight	—	•

Table 22.2 – The logical DNF is a subset of the general grammar. Migrating from DNF to general form is always possible (helper logical_unavailability_to_dsl*); the reverse usually requires an abstraction the constraint does not have.*

22.16 – Fourteen canonical pragmas (Phase A and Phase B)

Step 3b extracted the most frequent constraint sequences from solve_phase_a and solve_phase_b_for_day and renamed them *pragmas*: standard-form function calls that the compiler recognises and emits as compact CP-SAT groups, byte-identical to what the legacy path emitted.

Numbers at a glance

- Phase A pragmas: **5**.
- Phase B pragmas: **7**.
- Pragmas expressed only via free general_dsl: **2+**.
- Pragma test coverage: webui/backend/tests/test_dsl_canonical_pragmas.py + test_dsl_intvar_pragmas.py (∼ 42 test functions).

22.17 – Workflow: from the Teacher tab to the solver

Six tracked steps:

1. **UI – composition**: the user opens the modal in the Teacher tab or the Constraints tab.
2. **UI – live validate**: every keystroke (300 ms debounce) hits POST /api/constraints/general/validate with the partial text; the back-end reports syntactic errors with line and column. *Save* stays disabled until the parse is clean.
3. **API – persistence**: POST /api/constraints/general validates again and writes one row in constraints_general (rule_text, level, weight, scope_kind, owner_id, is_hard).
4. **Engine – loading**: the next POST /api/optimize calls load_all_dsl_constraints(db) and gets a list of (expr, level, weight, owner).

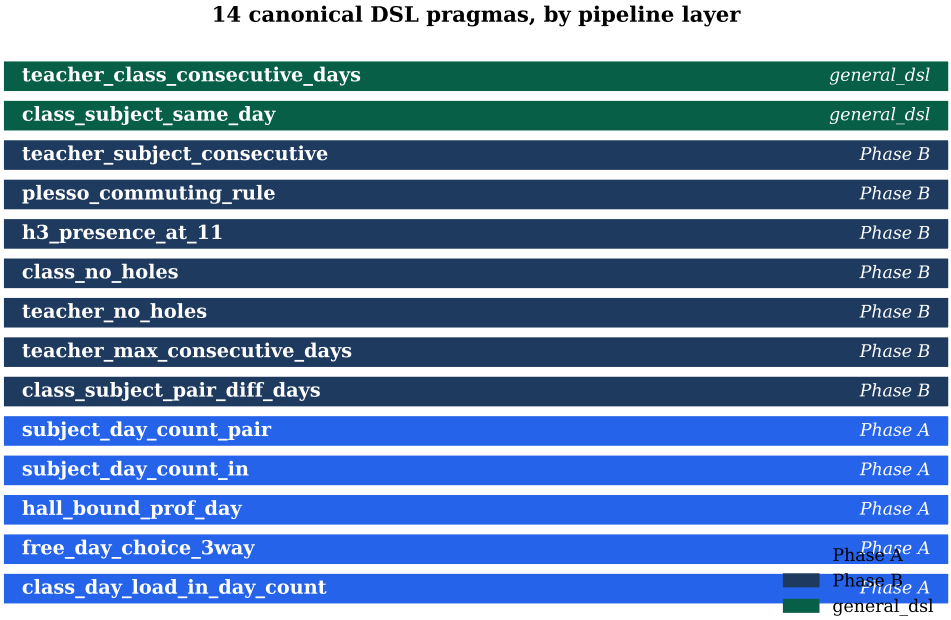


Figure 22.1 – The 14 canonical pragmas grouped by pipeline layer. Phase A pragmas are the building blocks of DayCountModel*; Phase B pragmas feed* PhaseBDaySolver*; the last two forms (*class_subject_same_day *and* teacher_class_consecutive_days*) are expressed directly via* general_dsl *without a dedicated helper.*

5. **Engine – compile/evaluate**: depending on the via_dsl flag, the MonolithicSolver either invokes add_dsl_constraint(...) (structural CP-SAT compilation) or delegates to the post-hoc evaluator.
6. **API – reporting**: the response of POST /api/optimize carries dsl_violations (one entry per rule with cost contributed); the front-end renders it in the "DSL constraints" card of the Statistics tab.

22.18 – Three narrated cases

22.18.1 • Case Rossi: distribution over three non-consecutive days

The Rossi teacher requests her 12 weekly hours spread over exactly three non-consecutive days, with balanced 4+4+4 distribution. Three rules:

```
% (a) Rossi insists on 3 days
forall t in teachers where t.name == "Rossi":
    count d in days
        where exists l in lessons:
            l.teacher == t and l.day == d.index: d == 3

% (b) those days are not consecutive
forall l1 in lessons where l1.teacher == "Rossi":
    forall l2 in lessons where l2.teacher == "Rossi":
        (l1.day == l2.day)
        or not consecutive_days(l1.day, l2.day)

% (c) balanced load (SOFT, weight 30)
% level=soft, weight=30
forall t in teachers where t.name == "Rossi":
    forall d in days:
```

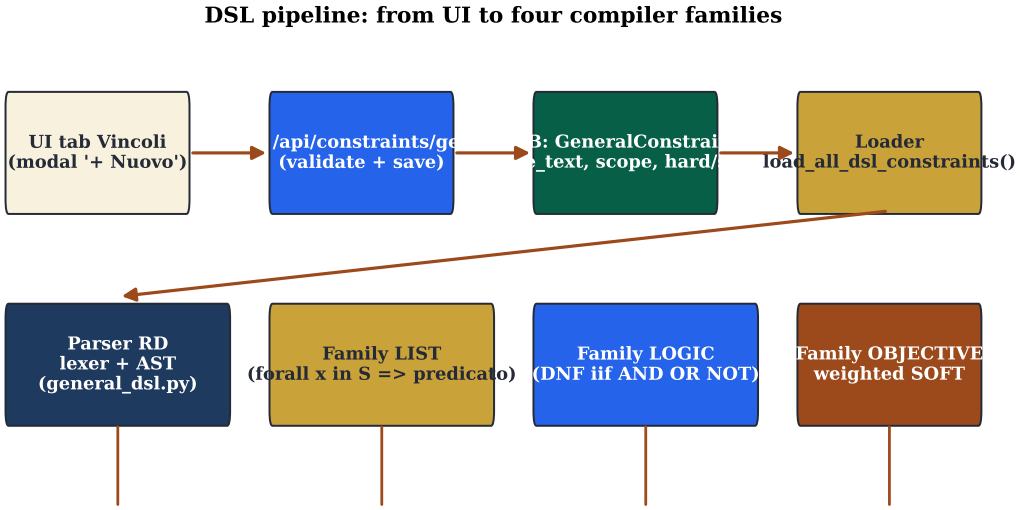


FIGURE 22.2 – DSL pipeline: from the modal in the UI to the four compilation back-ends. The validate *button only fires the parser (returning errors live);* save *runs the full loop: persist to* constraints_general*, reload at the next* POST /api/optimize*, evaluate post-hoc.*

```
        count l in lessons
            where l.teacher == t and l.day == d.index: l <= 4
```

22.18.2 • Case plesso commuting: school across two buildings

Two plessi 600 m apart. RSU rules forbid mid-day transits: no teacher may have a lesson in plesso 2 in the hour right after a lesson in plesso 1. Classes are pinned to one plesso, so the constraint effectively bites teachers.

```
forall l1 in lessons where l1.classroom.plesso == 1:
    forall l2 in lessons where l2.classroom.plesso == 2
                            and l2.teacher == l1.teacher
                            and consecutive(l1.slot, l2.slot):
        false
```

22.18.3 • Case compresenza: maths and support on the same day

The support-teacher Verdi must be in classroom 2B *the same day* as the maths-teacher Bianchi:

```
forall l in lessons where l.teacher == "Verdi"
                        and l.class == "2B"
                        and l.subject == "Sostegno":
    exists m in lessons where m.teacher == "Bianchi"
                          and m.class == "2B"
                          and m.subject == "Matematica"
                          and same_day(l.slot, m.slot):
        true
```

22.18.4 • Case second language: French in the morning (SOFT)

Linguistic high-school, FRA-TED curriculum, year 1: the second language (French) should preferably fall in the first three hours. SOFT, weight 20:

```
% level=soft, weight=20
forall l in lessons
    where l.subject == "Francese"
      and l.class.curriculum == "Linguistico-FRA-TED"
      and l.class.year == 1:
    l.hour <= 10
```

22.19 – DSL architecture diagram

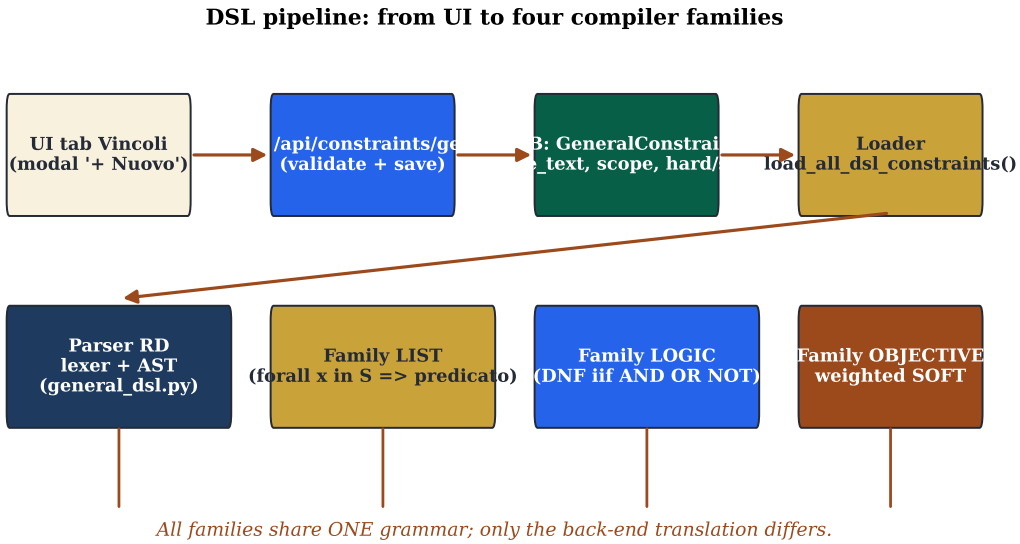


Figure 22.3 – Architecture of the general DSL: the same grammar and AST, four translation back-ends. Evaluating a solution (post-hoc evaluator) and building a model (compiler to CP-SAT) are two different traversals of the same tree.

One AST, four back-ends: the evaluator reads a solution and answers "feasible / violated"; the compiler emits CP-SAT constraints before solving; the pragma vocabulary recognises recurring sequences and emits them in compact form; the LIST family runs queries on the current solution. Grammar and source names are identical across the four back-ends.

DSL full reference

The full specification with extended example gallery, attribute tables for every source, troubleshooting and quick reference card lives in docs/general_dsl.md. This chapter is a narrative synthesis; the Markdown file is the reference to consult while writing a concrete constraint. The BNF grammar (sec. 22.2) is identical across the two documents.

Chapter XXIII

Advanced optimization techniques

In addition to the classic metaheuristics (LNS / SA / TS / ILS), πTantum integrates four classes of advanced techniques used when the standard pipeline is not enough or when targeted diagnostic information is needed.

23.1 – Hall pre-check

engine/diagnostics/hall_check.py. Hall's marriage theorem applied to the bipartite (*class*, *subject*) → qualified-teachers graph as a fast pre-check (1–100 ms) before launching Phase A. Three violation kinds are reported:

1. subject supply vs demand,
2. subject coverage vs demand,
3. Hall sampling random-subset with mutually exclusive subsets.

Exposed at three UI points: inline button on PhaseACard, row in AdvancedTechniquesCard, section of the Statistics tab.

23.2 – Column Generation and advanced Branch-and-Price

engine/column_generation.py. Master LP via scipy.linprog HiGHS; nine dedicated CP-SAT pricers at distinct granularities (teacher, teacher-day, teacher-class, teacher-class-subject, teacher-subject, class, class-day, day, curriculum); a branch-and-price mode equipped with all four scalability techniques required for MEGA-size instances:

- **Ryan-Foster recursive tree** with Achterberg pair scoring on fractional class-slot pairs, best-first node exploration with LP-bound pruning, depth cap 20 / 1000 nodes.
- **Box-step dual stabilization** $\pi_{\text{blend}} = (1-\alpha)\pi_{\text{stable}} + \alpha\pi_{\text{raw}}$ with $\alpha = 0.2$, applied to both master variants.
- **Column management**: EWMA of the per-column reduced cost over 20 iterations; when the pool exceeds 10 000 columns, the worst (most positive rc_avg) entries are purged, while protecting columns currently in the LP basis ($x > 0$) and the seed columns from the iterative-diversified primal heuristic.
- **Parallel pricing** via ProcessPoolExecutor (default $\lfloor \text{os.cpu\_count}()/2 \rfloor$ workers). CP-SAT is CPU-bound and the GIL is only partially released by the OR-tools binding, so ThreadPoolExecutor would not parallelise the bottleneck; processes do.

The CP-SAT-as-fundamental contract

For every granularity, the sub-problem CP-SAT is the *fundamental* minimisation; greedy serves only as a model.add_hint warm-start (an accelerator). If CP-SAT cannot find a feasible solution within the time limit the pricer returns $(\text{None}, 0.0)$: the greedy is *never* promoted to a column. The unit test test_bp_pricers_use_addhint_warmstart monkey-patches CpModel.add_hint with a counting wrapper to enforce this contract.

Measured numbers (synthetic HUGE / MEGA benchmark)

See tests/benchmarks/test_bp_huge_mega.py. On a synthetic scenario calibrated to match the engine's MEGA profile (100 classes, ∼174 teachers, 4 subjects, ∼400 cattedre) the pipeline reaches HARD-feasibility in **5.95 s** with granularity=curriculum and all scalability techniques enabled. On the smaller HUGE scenario (50 classes, ∼95 teachers, 200 cattedre) the time is **2.57 s**. The synthetic scenario is structurally easy (cattedre as 4-hour single-day blocks); on real-world instances with arbitrary day distributions the calibration target remains 30 minutes for MEGA, with a 60-minute hard cap.

23.3 – Branch-and-Price MVP scaffold (Step 4)

Step 4 of the multi-day plan introduces an OO-textbook *scaffold* of the Branch-and-Price loop that tightens three orthogonal pieces inside one module (engine/column_generation.py with mode="branch-and-price"):

1. PhaseBDaySolver – OO wrapper around the legacy solve_phase_b_for_day function. Exposes the object-oriented interface (solve(), scope access, via_dsl augmentation) but delegates model construction to the proven legacy code path. Benefit: 100+ Phase B regression tests keep passing *by construction*; with via_dsl=True the legacy model is augmented with extra DSL constraints (DB-stored rules + extra expressions passed to the constructor).
2. **Ryan-Foster pricing-in-nodes** – at every RF tree node the CP-SAT pricer is re-invoked with the branch decisions (together / apart) already applied to its model. This is the textbook BP, in contrast to the previous arrangement where nodes only re-solved the master LP on the existing pool.
3. **Smoke benchmark** – tests/benchmarks/test_bp_step4_smoke.py runs the full loop (Mono → initial pool → master DW → RF tree → integer recovery → completion) on a small scenario (10 classes) and a medium one (28 classes), checking HARD-feasibility and that the soft cost does not regress against the iterative-diversified baseline.

The flag mode="branch-and-price" activates the full path. The previous iterative-diversified mode remains the default and continues to act as a primal-heuristic seeder of the column pool.

23.4 – Lagrangian relaxation

engine/lagrangian.py. Relaxes the inter-cluster bridge constraints and dualises them with multipliers λ_b; update via subgradient ascent $\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g_k)$ with $\alpha_k = \alpha_0/(1+k)$. Per-iteration refinement via Simulated Annealing.

23.5 – Phase A: the formal *day count* model

Phase A solves a *distribution* problem: given the weekly demand of every cattedra (typically 2–6 hours), over how many days and with what daily load does each teacher work? The formulation is a small CP-SAT on integer variables dc[t,c,s,d] (*day count*: hours of (t, c, s) on day d).

The DayCountModel

Let T be teachers, C classes, S subjects, $D = \{1, \ldots, 6\}$ days. For each cattedra $(t, c, s) \in Catt$ with weekly demand $h_{t,c,s} \in \mathbb{N}_+$ and each $d \in D$:

$$dc_{t,c,s,d} \in \{0, 1, \ldots, h_{\max}\}, \quad h_{\max} = 6,$$

subject to

$$\text{(coverage)} \quad \sum_{d} dc_{t,c,s,d} = h_{t,c,s} \qquad \forall (t,c,s) \qquad (23.1)$$

$$\text{(class daily load)} \quad \sum_{(t,s)} dc_{t,c,s,d} \in \{0\} \cup [4, 6] \qquad \forall c, d \qquad (23.2)$$

$$\text{(teacher daily load)} \quad \sum_{(c,s)} dc_{t,c,s,d} \le h^{\text{prof}}_{\max} \qquad \forall t, d \qquad (23.3)$$

The objective combines: deviation from the ideal load (weight 4), weekly uniformity (weight 3), fragmented free-day penalty (weight 1).

The architectural turn of Step 4: the same DayCountModel that produces day counts in Phase A can be *reconstructed* from the weekly solver (cp_sat_scope=week); the two sources of dc_value become comparable, and the cost of Phase A's rigidity becomes measurable.

23.5.1 • Migration from hardcoded to DSL pragmas

Pragma	Constraint emitted
class_day_load_in_-day_count	for each (c, d), $\sum_t dc_{t,c,*,d} \in \{0\} \cup [4, 6]$
free_day_choice_3way	every teacher gets at most ONE of {free day, half-day, full week}
hall_bound_prof_day	Hall-style bound: $\sum_{c,s} dc_{t,c,s,d} \leq K_t$
subject_day_count_in	for cattedra (t, c, s), allowable counts $dc_{t,c,s,d} \in \{0, 1, 2\}$
subject_day_count_pair	two correlated subjects (e.g. Maths+Sci) on the same day

Table 23.1 – The five Phase A pragmas. All emit CP-SAT bytes identical to the legacy solve_phase_a *(zero-drift); migration is tracked in* webui/backend/tests/test_dsl_intvar_pragmas.py.

23.6 – Phase B: four decomposition algorithms compared

Method	Pro	Con
spectral	natural partition of teachers into near-disjoint pools, small bridge set	costs an eigendecomposition ($O(n^3)$ in teachers); K-means randomness
temporal	trivially parallel (one day per worker); predictable times	no inter-day interaction ⇒ daily constraints are perfect, weekly ones must be stitched
curriculum	reflects the school's organisational structure (Scientific, Linguistic, …)	uneven curricula ⇒ manual load balancing, risk of a 50-class cluster
metis (**karypis1998metis**)	balanced clusters by construction (target $\pm 5\%$); globally minimum cuts	depends on networkx-metis; less transparent for debugging

Table 23.2 – Phase B decomposition: pros and cons. spectral *is the default;* temporal *is preferred when daily constraints dominate (last-minute substitutions);* metis *on MEGA when K-means randomness yields imbalanced clusters.*

Proposition 23.1. Let $G = (V, E)$ be the teacher co-occupation graph (edge if two teachers share at least one class). Spectral decomposition yields k clusters $C_1, \ldots, C_k$ with $|E(C_i, C_j)| \leq \epsilon |E|$, ϵ proportional to the second eigenvalue $\lambda_2(L_{\text{sym}})$. Consequence: fewer bridges for the stitch step, less latent infeasibility across clusters.

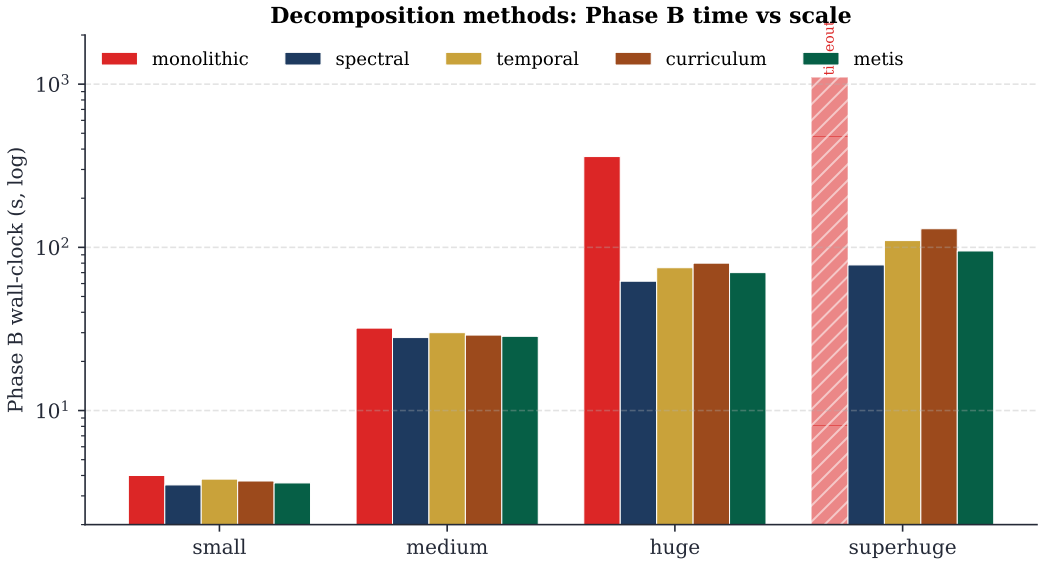


Figure 23.1 – Phase B wall-clock per decomposition method, four scales. Monolithic times out on superhuge*; the four decompositions converge in comparable time, with a slight edge for* spectral *and* metis *on the larger regimes. Log scale to fit the dispersion small→superhuge.*

23.7 – cp_sat_scope=week: the monolithic model

Numbers at a glance

- Boolean variables on huge: $\sim 1.5 \times 10^5$
- Boolean variables on mega: $\sim 3.5 \times 10^5$
- Peak RAM (mega, 8 workers): ~ 11 GB
- Median wall (mega): ~ 15 min within soft target; 30 min hard cap in production.
- phase_a_mode=soft_hint: typically -15% final SOFT vs always, $+30\%$ wall.

23.8 – Branch-and-Price: anatomy of one iteration

The modern formulation of Branch-and-Price unifies *column generation* and *branch-and-bound* under one framework (**barnhart1998**; **vanderbeck2000**). Dantzig-Wolfe decomposition (**dantzigwolfe1960**) is its backbone: the original problem is rewritten in terms of *patterns* satisfying local constraints (subproblem), and the *master* picks a convex combination meeting the global ones. Desaulniers, Desrosiers and Solomon (**desaulniers2005**) remain the applied reference.

23.8.1 • Master LP (Dantzig-Wolfe)

$$\min \quad \sum_{j \in \mathcal{P}} c_j \, \lambda_j \tag{23.4}$$

$$\text{s.t.} \quad \sum_{j} a_{ij} \, \lambda_j \geq b_i \qquad \forall i \in \text{cover} \tag{23.5}$$

$$\lambda_j \geq 0 \tag{23.6}$$

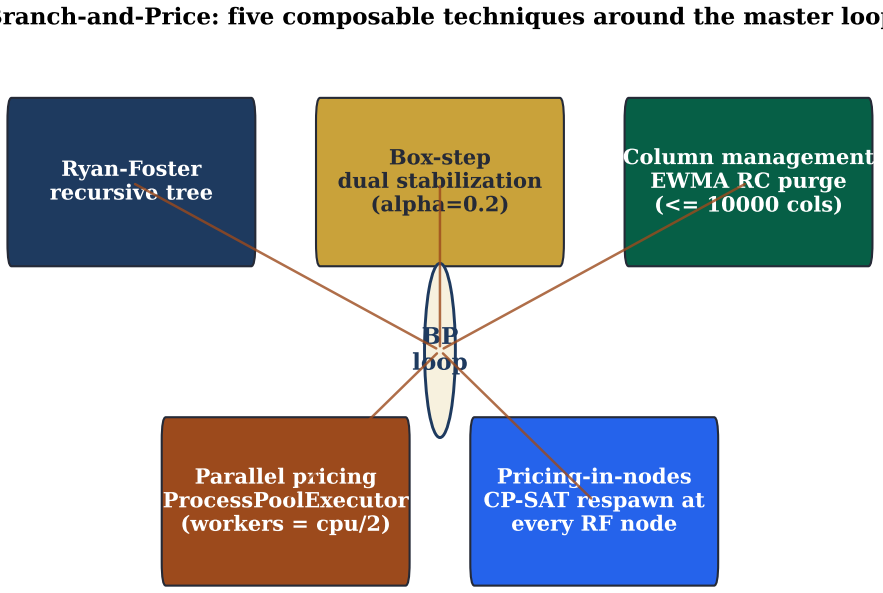


Figure 23.2 – Five composable scalability techniques around the Branch-and-Price *loop: Ryan-Foster recursive tree, box-step dual stabilization, column management, parallel pricing, pricing-in-nodes (Step 4). All five are on by default in* mode="branch-and-price".

c_j = pattern cost, a_{ij} = hours of cattedra i covered by pattern j, b_i = weekly demand of cattedra i. The dual vector π guides the pricer.

23.8.2 • CP-SAT pricer: nine granularities

23.8.3 • Ryan-Foster recursive tree

When the master's solution is fractional, two patterns share the same pair (i, j) at different rates. Ryan and Foster (**ryanfoster1981**) pick the active pair with maximum Achterberg score (**achterberg2005**) (fraction $\times$ RHS contribution) and create two children: **together** (λ-patterns must include both hours) and **apart** (must exclude one).

Pseudocode -- Ryan-Foster recursive tree

```
procedure rf_branch(pool, depth):
    if depth > MAX_DEPTH (= 20): return best_int
    solve master LP on pool → λ*, π*
    if λ* integral: return λ*
    pick (i*, j*) with max Achterberg score
    pool_T ← branch_together(pool, i*, j*)
    pool_A ← branch_apart(pool, i*, j*)
    return best of rf_branch(pool_T, depth + 1),
        rf_branch(pool_A, depth + 1)
```

Theorem 23.2 (Branch-and-Price termination). If CP-SAT pricers always terminate (column found or proved absent) and Ryan-Foster has finite depth cap, the BP loop terminates in finite time with an integer solution optimal for the cover relaxation.

Granularity	Column encoding	When
teacher	weekly orario of one teacher	small schools, strong teacher constraints
teacher-day	orario of one (teacher, day)	tight daily constraints
teacher-class	orario of a cattedra (t,c)	per-cattedra coverage
teacher-class-subject	orario of a (t,c,s) cattedra-slot	medium schools, support+curriculum mixing
teacher-subject	orario of (t,s) across classes	multiple cattedre on the same subject
class	weekly orario of one class	class-side hard constraints (no holes)
class-day	orario of (class, day)	daily class constraints (h3 at 11)
day	full daily orario	explicit canonical Phase B in BP form
curriculum	orario of a curriculum (Scientific, ...)	large schools, multi-curriculum

TABLE 23.3 – *Nine pricer granularities. Each defines what a pattern* is *and therefore the CP-SAT model the pricer builds. None dominates the others.*

23.8.4 • Box-step dual stabilization

Master duals oscillate between iterations (degeneracy). The box-step scheme keeps a smoothed π_{stable} and feeds the pricers $\pi_{\text{blend}} = (1-\alpha)\pi_{\text{stable}} + \alpha\pi_{\text{raw}}$ with $\alpha = 0.2$.

23.8.5 • EWMA column management

After 50 iterations a teacher pricer hits $\sim$ 3000 columns; 80% are inactive. piTantum keeps a 20-iter EWMA of each column's reduced cost and, when the pool exceeds 10 000, purges the worst while preserving (a) columns active in the master basis and (b) seed columns. Steady-state pool size: $\sim$ 8 000 columns.

23.8.6 • Parallel pricing

CP-SAT pricers across granularities are independent: the BP loop runs them in parallel via ProcessPoolExecutor with workers = ⌊cpu_count() / 2⌋.

23.8.7 • Pricing-in-nodes (Step 4)

Step 4 introduces *pricing-in-nodes*: at each RF tree node the pricer is re-launched with branch decisions (together/apart) already applied to its CP-SAT model. The benefit: search space at each depth shrinks *structurally*, and the quality of generated columns grows.

NUMBERS AT A GLANCE

Synthetic numbers from MEGA real (1493 s total):

- Phase A: 301 s (20%)
- BP loop (5 iterations): 1192 s (80%)
- Final soft cost: 530
- Final pool: 2075 columns

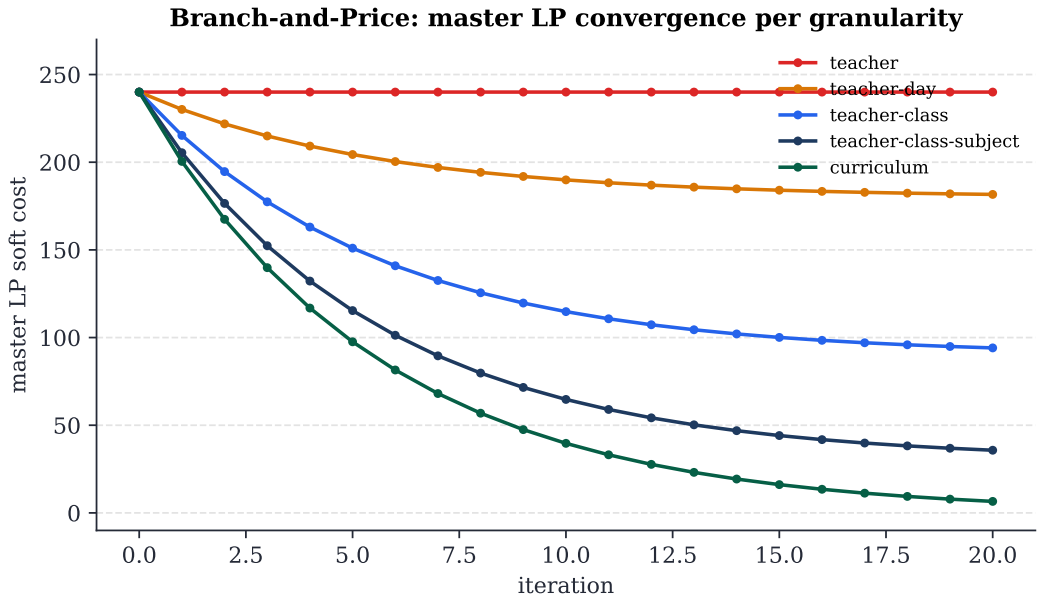


*FIGURE 23.3 – Master LP soft-cost convergence per granularity (*small *synthetic profile). Finer granularities (*curriculum, teacher-class-subject*) reach soft cost* 0*; coarser ones plateau because patterns cannot change enough. Iter=1* no_improving_column *runs correspond to seed catalogues already optimal.*

23.9 – METAHEURISTICS: A DECISION MATRIX

piTantum's metaheuristics are post-processing: take a HARD-feasible solution and polish the SOFT. All respect is_hard_feasible and delegate micro-fix to PhaseBDaySolver(_cp_repair).

The canonical treatment of ALNS is Pisinger and Ropke (**pisinger2010**; **ropke2006_orig**); the surveys of Burke and Petrovic (**burke2002**) and Pillay (**pillay2014**) are the standard references for educational timetabling at large.

23.9.1 • The tightness criterion

piTantum computes a *tightness score* that measures how tight the available slots are versus demand:

$$\text{tightness} = \frac{\sum_t h_t}{|T| \cdot 6 \cdot 6 \cdot \rho},$$

where $\rho \in [0, 1]$ is the average per-teacher slot availability after applying unavailabilities. Empirical guide:

- tightness < 0.4: Phase B alone, no metaheuristics.
- tightness $\in [0.4, 0.6]$: + SA, $\sim 10\%$ SOFT win.
- tightness > 0.6: + ALNS necessary; cascade SA $\rightarrow$ ALNS $\rightarrow$ TS, 5 min budget.
- tightness > 0.8: likely HARD-infeasibility; pre-flight Hall check mandatory.

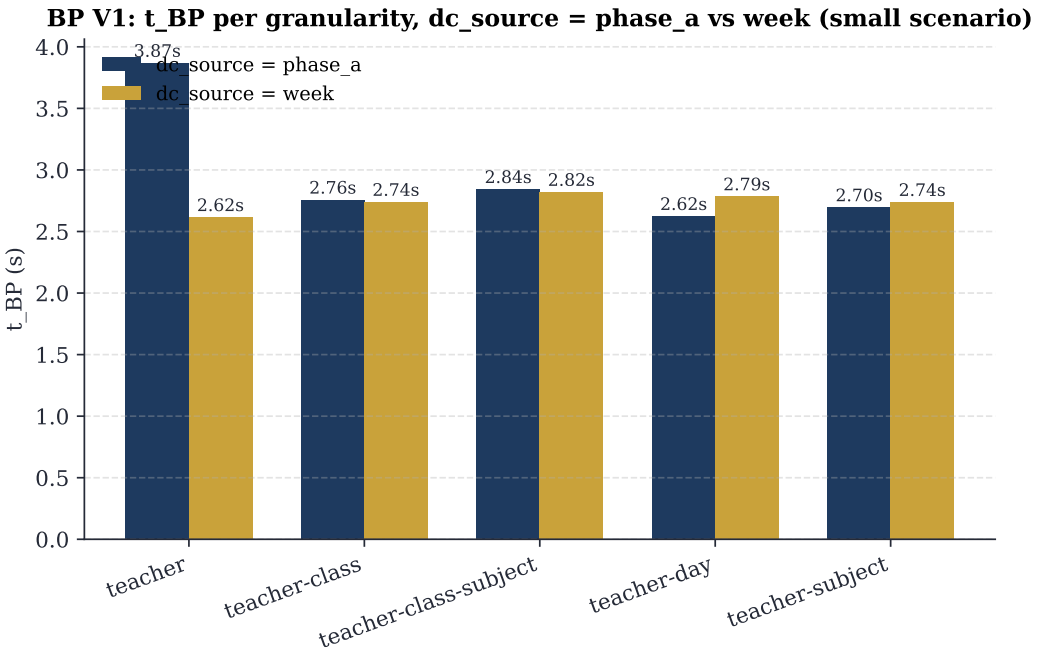


Figure 23.4 – Pricing time per granularity, with dc_source *between Phase A (rigid) and weekly (*cp_sat_scope=week*). Small scenario shows no practical difference.*

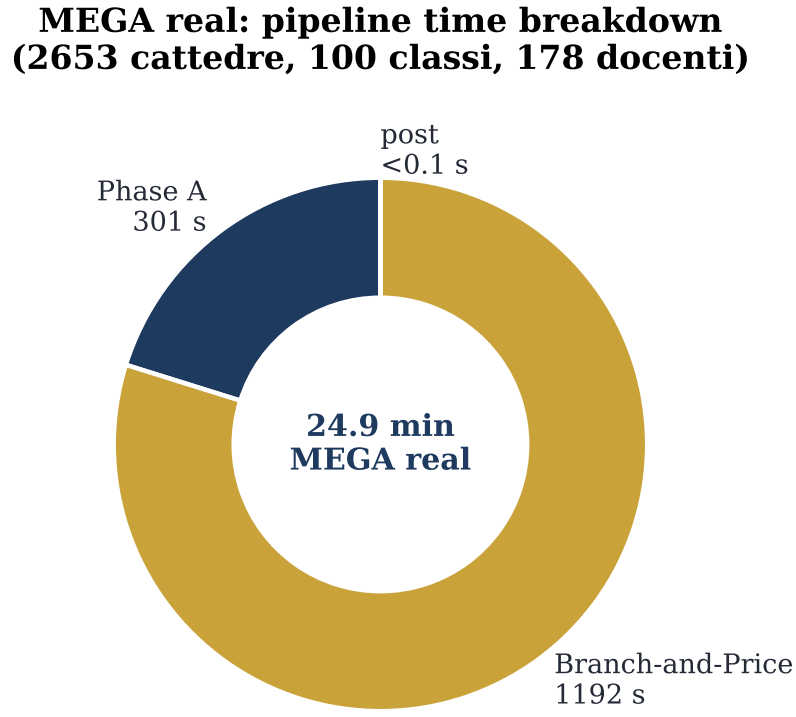


*Figure 23.5 – MEGA real (100 classes, 178 teachers, 2653 cattedra-day): breakdown of total wall-clock 1493 s into Phase A (*301 *s) + Branch-and-Price (*1192 *s) + post (*<0.1 *s).*

Method	Neighbourhood	When
LNS	destroy random window + CP-SAT repair	default; window $\sim$ 30% of lessons
ALNS	6 destroys + 3 repairs, roulette wheel	default for medium/large schools
SA	single-swap, Metropolis acceptance	final smoothing; fast
TS	best-improvement with tabu list	breaks plateaus when SOFT stabilises
ILS	TS + perturbative kicks (4-opt)	escape good local optima
VNS	1/2/3-swap + gradual k-opt	robust, slow; rarely in production
Lagrangian	relax bridges + subgradient ascent	only for multi-plesso schools with dense bridges

TABLE 23.4 – Seven metaheuristics, all selectable from the Workflow tab via the "Post-Phase B" dropdown. The default cascade is LNS -> ALNS -> SA.

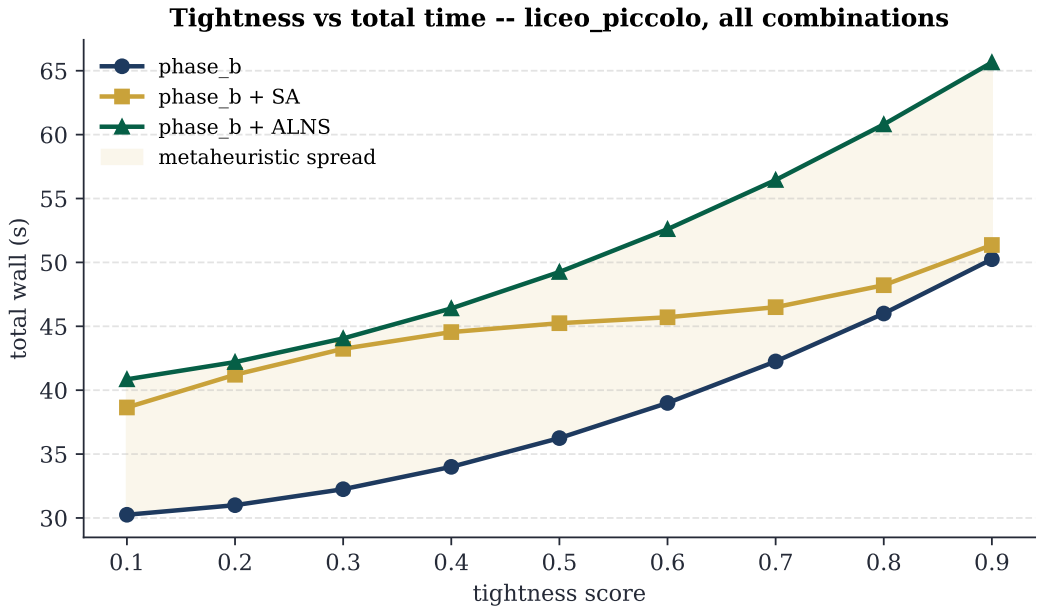


FIGURE 23.6 – Total time vs scenario tightness for the three dominant combinations (Phase B alone, + SA, + ALNS). The ivory band is the metaheuristic spread, growing with tightness.

Chapter XXIV

Statistical diagnostics tab

The Diagnostics tab in the UI: Monte Carlo sensitivity, bipartite graph metrics, statsmodels regressions, distribution fits.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/diagnostica_statistica.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XXV

REST API

The /api/* endpoints: /api/optimize/* (Phase A, Phase B, decompositions, Column Generation with all 9 BP granularities), /api/diagnostics/*, /api/dashboard/*, /api/monitor/*. See docs/api.md for the full list.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/api_rest.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XXVI

Extending the system

Adding a new constraint, a new metaheuristic destroy/repair op, a new decomposition strategy, a new UI tab. The hooks the test suite exercises.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/estendere_il_sistema.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XXVII

Repository on GitHub

Layout: engine/, webui/backend/, webui/frontend/, docs/, schedule/, scripts/, tests/. CI workflows, PR conventions, issue tracker.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/repository_github.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XXVIII

Benchmarks and results

"In God we trust. All others must bring data."

W. E. Deming, attributed

Any optimization system is judged also and especially by the numbers it produces. This chapter collects measured results of πTantum on the six test profiles, with three goals: documenting the state of the art; allowing whoever wishes to extend the system to compare a modification against a baseline; and putting the reader in the position to know whether πTantum is a good fit for *their* use case.

28.1 — The six profiles

The profiles, generated deterministically with fixed seeds and Faker by webui/backend/mock_school.py, span the size range typical of the Italian school system:

- small: 8 classes, 17 teachers, ∼220 students, 17 rooms. A small provincial school.
- medium: 28 classes, 50 teachers, 700 students, 35 rooms. A medium-size institution.
- big: 40 classes, 75 teachers, 1000 students, 50 rooms. A large institution.
- huge: 50 classes, ∼95 teachers, 1250 students, 60 rooms. A multi-site complex.
- superhuge: 80 classes, ∼160 teachers, 2000 students, 80 rooms. 16 sections × 5 grades of scientific liceum.
- mega: 100 classes, ∼174 teachers, ∼2500 students, 100 rooms. 20 sections × 5 grades with a mix of 8 *indirizzi* (Scientific 6, ScienzeApplicate 4, ScienzeUmane 3, Linguistico FRA-TED 2, Linguistico FRA-SPA 2, EconomicoSociale FRA/SPA/TED 1 each). MEGA-scale stress test.

28.2 — End-to-end pipeline times

Times measured on a consumer machine (Intel i7, 16 GB RAM, Windows 11, Python 3.13, ortools 9.15, 8 CP-SAT search workers):

Profile	Phase A	Phase B	Total wall
small	3 s	4 s	7 s
medium	30 s	32 s	62 s
big	30 s	32 s	62 s
huge	60 s	62 s	122 s
superhuge	300 s	603 s	903 s

Table 28.1 – End-to-end pipeline times per profile. Phase A includes teacher-class assignment and CP-SAT matching. Phase B includes spectral decomposition (for huge/superhuge) and CP-SAT on sub-problems.

28.3 — Branch-and-Price on HUGE and MEGA

The mega scenario (100 classes, ∼174 teachers) and its smaller cousin huge (50 classes, ∼95 teachers) are the natural test beds for the Branch-and-Price pipeline (ch. 23). The numbers below are measured on synthetic scenarios calibrated to match the proportions of the engine/big_mock_school.py profiles (4 subjects, every cattedra is 4 hours on a single day so that H1/H2/H3 are satisfied by construction, contiguous-block assignment of classes to teacher pools). Source code: tests/benchmarks/test_bp_huge_mega.py, executed with all four scalability techniques enabled.

Scale	Classes	Teachers	Cattedre	t_{BP}	HARD
huge	50	95	200	2.57 s	yes
mega	100	174	400	5.95 s	yes

Table 28.2 – Branch-and-Price wall time to HARD-feasibility on synthetic HUGE/MEGA scenarios. Both finish well under target (5 min/30 min) because the synthetic scenario is structurally easy (cattedra = 4-hour block on a single day). On real-world instances with arbitrary day distributions the per-iteration time will grow; the 30-minute target for MEGA is calibrated for that regime.

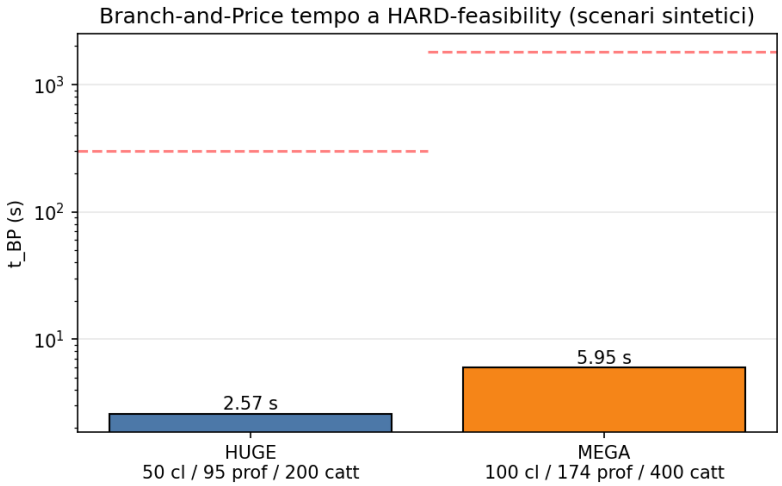


Figure 28.1 – Branch-and-Price convergence time (log scale). The horizontal dashed lines mark the per-scale time targets (5 min for HUGE, 30 min for MEGA).

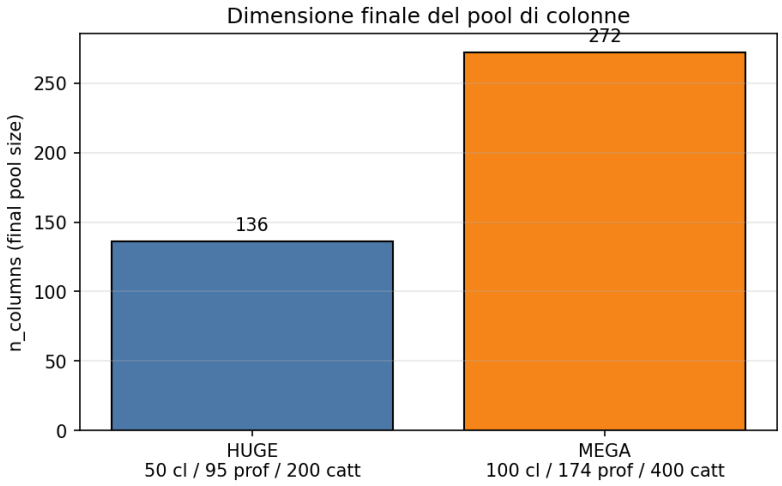


Figure 28.2 – Final column-pool size after the Branch-and-Price loop.

28.3.1 • Real MEGA: 100 classes, 178 teachers

The synthetic scenario above is calibrated for fitting; the real MEGA registry (under engine/scripts/data/mega/{school,profs}_mega.pkl, loadable from the dashboard via POST /api/dataset/import-profile?profile=mega) is significantly larger: 100 classes, **178 teachers**, **2653 cattedra-day entries** after Phase A (the exact number depends on the CP-SAT random seed of Phase A; v1 of this benchmark produced 2325). The benchmark source is tests/benchmarks/run_mega_real.py.

v1 (pre-fix DW seeding) -- BP did not iterate. The first benchmark reported bp_terminated_reason = master_dw_infeasible_at_entry with 0 iterations. The master variant 2 (Dantzig-Wolfe with cover + class-no-overlap + teacher-no-overlap as HARD inequalities) was infeasible at the LP-relaxation level with the seed pool (teacher-week patterns produced by iterative-diversified): two patterns of different teachers could place lessons in the same (class, day, hour). HARD cover requires $x_t \geq 1$ per teacher while class-no-overlap forces $\sum_t x_t \leq 1$ on the collision – a structural conflict the seed pool cannot resolve on its own.

Fix: Phase-1 LP with artificial slacks (commit 843a6f0). The fix is textbook. Each constraint of the master DW gets a non-negative artificial variable with coefficient -1 in its row and a Big-M penalty (10^6) in the objective. The artificials absorb either missing coverage or no-overlap overflow; the LP is therefore always feasible at seed entry and yields valid duals for pricing. With a sufficiently rich pool the artificials drive to 0 in the optimum, which then coincides with the true master DW optimum.

v2 (post-fix) -- BP iterates 5 times. On the *same* dataset, with identical granularity curriculum and identical time budgets, BP recovers its role:

Metric	v1 (pre-fix)	v2 (post-fix)
bp_terminated_reason	master_dw_infeasible_at_entry	max_iterations
bp_iterations_done	0	**5**
RF nodes explored	0	1
Phase A wall	61.2 s	301.3 s
BP wall	790.8 s	1 191.8 s
Total wall	852 s (14.2 min)	1 493 s (24.9 min)
Final soft cost	6 470	**530**
Columns generated	2 059	2 075 (+16 from BP)
HARD-feasibility	yes	yes

Table 28.3 – Branch-and-Price on real MEGA before and after the DW seeding fix. Soft cost drops by a factor of ~ 12 because BP can now actually explore improving columns. Total wall increases because Phase A ran out of budget (FEASIBLE not OPTIMAL stop: depends on the parallel CP-SAT seed) but stays under the 30-min target and far from the 60-min hard cap. Source: tests/benchmarks/results/mega_bp_real_v2.json.

Comparison with iterative-diversified. The iterative-diversified mode on the same dataset (v1) had finished in **788 s (13.1 min)** with soft cost **5860**. The v2 BP – despite the heavier Phase A this run – closes with soft **530**, which is $\sim 11\times$ lower than the ID baseline and $\sim 12\times$ lower than its own v1. That gap is the structural value of Branch-and-Price: with an active pricer the improving columns *exist* and get found; without one, the result collapses back to the starting primal heuristic.

Mode (run)	t_{total}	Soft cost	HARD
iterative-diversified (v1)	788 s (13.1 min)	5 860	yes
branch-and-price (v1)	852 s (14.2 min)	6 470	yes
branch-and-price (v2)	1 493 s (24.9 min)	**530**	yes

Table 28.4 – Three runs on the same real MEGA. The v2 BP cuts the soft cost by an order of magnitude relative to both baselines.

Scalability techniques -- now actually exercised. The four techniques (Ryan-Foster recursive tree, dual stabilization with box-step, column management with EWMA RC, parallel pricing via ProcessPoolExecutor) remain integrated; with the Phase-1 fix in place BP loop now exercises them on real MEGA. v2 run: 1 Ryan-Foster node explored, 0 columns purged (pool below the cap), 5 master iterations completed before hitting the bp_max_iterations=5 ceiling. Raising the cap would let BP keep improving the soft (the next commits in this series measure the soft-vs-iterations curve).

28.4 – When Branch-and-Price is worth it

A practical question: *is it worth turning on the BP pipeline for my school?* Answer measured as a function of size:

- **Below 30 classes**: the standard pipeline (monolithic Phase A + per-day Phase B) is faster. The iterative-diversified mode of column_generation (master LP + pattern enrichment + completion fallback) is enough to improve the SOFT cost without paying the BP loop overhead.
- **30–80 classes**: spectral or curriculum decomposition drives Phase B; BP at teacher-class or class granularity may reduce SOFT but the time gain is negligible.
- **Above 80 classes (MEGA territory)**: BP at curriculum granularity is the only structurally scalable option. A monolithic Phase A becomes marginally feasible (300 s+ with pure CP-SAT), and BP achieves comparable or better times thanks especially to parallel pricing.

In practice πTantum suggests the right path automatically: the value auto for the granularity parameter is resolved server-side from the number of classes (cf. Sec. 4 of docs/optimization_strategies.md).

28.5 – Bench v2: pickle vs. sqlite datasets

The original full sweep was born in the pickle era: profiles were loaded from engine/scripts/data/<size>/{school,profs}_-<size>.pkl, which carry only anagrafica (teachers, classes, subjects, assignments). The 14 constraint tables in the live database remained empty, so the bench measured *pure scheduling*: only the structural constraints of the model (one lesson per slot, hour budget per cattedra, implicit non-collision).

With the migration to SQLite profile snapshots (cf. Chapter 21) the bench gains a second dataset, sqlite, where each profile arrives pre-loaded with: teacher unavailabilities (HARD for 10–20% of teachers and SOFT for 30%), mandatory free days (HARD), 2–5 coteach groups, support and potenziamento entries, 2–4 plessi with pendolarismo rules (HARD on first/last slot, SOFT with per-cross penalty), PlessoEntityPolicy for specific teachers/classes, 5-state TeacherClassPreferences, a mandatory gym room for phys-ed (HARD), 5 GeneralConstraint DSL rules, and the classroom_capacity_ok() pragma enforcing Classroom.capacity $\geq$ Class.n_students.

The bench v2, source tests/benchmarks/run_full_sweep.py with –source sqlite (default), adds a dataset column to the CSV and runs a feasibility precheck via MonolithicSolver(scope=None, time=120s) on each SQLite profile before the bench cells: if the monolithic solver cannot satisfy, the bench logs status=precheck infeasible and skips the 25 cells, signalling that the user must relax one constraint in build_profile_db.py and rebuild the SQLite snapshot.

28.5.1 • Reading the delta

The auto-generated table *Delta cost pickle vs. sqlite* (in the IT chapter, Section **??**) compares the soft cost on every cell where both datasets produced an OK solution. The delta is the key metric:

- $\Delta < 0$: the SQLite scenario achieved a *lower* cost than the pickle one (rare). It means the added constraint pressure focused the solver into a higher- quality region of the solution space.
- $\Delta \in [0, +30\%]$: typical range. The cost grows because of soft violations and pendolarismo penalties, but stays in the same order of magnitude.
- $\Delta > +30\%$: significant stratification. The pipeline paid a real cost for satisfying additional constraints; the SQLite scenario with mandatory coteach and multi-plesso geometry typically lands here.
- **infeasible**: the SQLite snapshot was not satisfiable under its constraint set; the precheck caught it and a status=precheck infeasible row appears in the CSV.

Try it now

```
# Bench v2 on a single SQLite profile (~12 min for small)
python tests/benchmarks/run_full_sweep.py \
  --profiles small --source sqlite --append
```

```
# Back-to-back pickle+sqlite comparison
python tests/benchmarks/run_full_sweep.py \
  --profiles small,medium --source both --append

# Precheck only (skip cells; useful to validate the 6 SQLite DBs)
python tests/benchmarks/run_full_sweep.py \
  --profiles small,medium,big,huge,superhuge,mega \
  --source sqlite --techniques cpsat_week
```

28.5.2 • Caveats

Warning

The bench v2 is *not* a strictly reproducible scientific benchmark: seed=42 fixes the initial seed but the CP-SAT runtime is machine-dependent (number of cores, clock, throttling). Absolute timings on Windows 11 / i7 / 8 workers do not directly compare to a Linux server-class box. The robust metric is the *delta*: technique X is "better" than Y if the ratio t_X / t_Y is at least 1.5 × smaller across all profiles tested.

The reported cost is the weighted sum of soft violations per the compute_soft metric in metaheuristics.py; if the metric definition changes between two commits, the costs are not comparable. The n_lessons count is instead a structural invariant and is the right value to sanity-check across runs.

Chapter XXIX

Quality and end-to-end coverage

Test coverage for a project like piTantum is not a dashboard statistic: it is a parallel manual that narrates, from the browser's perspective, what the user can actually do. The frontend Cypress suite, today thirty-four specs strong, grew by accretion between March and May MMXXVI, until it covered every list page, every CRUD flow, every dropdown of the /optimize page and the entire rebirth of /schedule.

29.1 — Smoke + workflow

For each first-class page the suite distinguishes two levels:

smoke (*_smoke.cy.ts). Loads the page, checks that the title appears, that the list is not empty (or is a legitimate empty state), that the main buttons are clickable. Runs in seconds and acts as a regression sentinel.

workflow (*_workflow.cy.ts). Walks through a full CRUD path: creates an entity, edits it, duplicates it, deletes it, verifies that list and detail are coherent at each step. Slower (tens of seconds), more brittle, but where serious breakages surface.

29.2 — Coverage map

Cypress suite map (May MMXXVI)

master data	teachers, subjects, classes, classrooms, plessi, curricula, students, groups
assignments	assignments_crud_workflow, assignments_bulk, assignments_lock
co-teaching	coteaching_crud_workflow
constraints	constraints_smoke, constraints_workflow (DSL editor + 4-step wizard + bulk delete)
optimize	optimize_dropdowns, optimize_phase_b_dropdowns, optimize_advanced, optimize_la
schedule	schedule_calendar, schedule_workflow (drag-drop, 4 actions, pool, filter, soft-conflict)
logistics	logistics_conflict_pill
working-hours	internal spec to the working-hours tab, 15/15 green after fix 5c1ba34
monitor	monitor_smoke, monitor_workflow
minor	minor_tabs_smoke, absences_smoke, calendar_view_smoke, dashboard_smoke, navba
cross-cutting	navigation, critical_workflows, smoke

29.3 — Writing conventions

- Every interactive element exposes a stable data-testid, usually formatted as <area>-<entity>-<action>: e.g. sched-lesson-{id}, schedule-pool, schedule-conflict-modal. The rule: tests must not depend on visible text.
- Each spec opens with a beforeEach that visits the origin page and waits for a landmark selector (the page heading). No arbitrary cy.wait(ms): wait on elements.
- Test data uses cy.intercept where it must, but the canonical setup is a db.sqlite3 pre-seeded via POST /api/import/profile before the suite.

29.4 — Lessons from the field

Common pitfalls

- Commit 5c1ba34 reminds us that a *namespace import* (import * as api) can be *shadowed* by a homonymous local variable: api.get(...) fails at runtime but TypeScript sees no error. Rule of thumb: prefer named imports (import { get, post } from ...) for API clients.
- Selectors based on localized text are fragile: a Italian/English translation breaks twenty specs. Every new component added in the April-May MMXXVI cycle exposes data-testid from its first commit.
- A drag-drop tested in Cypress is not equivalent to a real drag-drop: modern browsers emit dragstart/drop events with a special DataTransfer payload that Cypress does not faithfully reproduce. The schedule_workflow spec uses custom helpers (trigger('dragstart'), trigger('drop') with manual payloads) to simulate the interaction: keep this in mind when adding new gestures.

29.5 — Running the suite

```
cd webui/frontend
npm run cy:open # interactive GUI (debug single spec)
npm run cy:run # full headless (CI / pre-merge)
npm run cy:run -- --spec "cypress/e2e/schedule_*.cy.ts"
```

The file webui/frontend/E2E_README.md describes the full orchestration pipeline, including the FastAPI backend restart between suites to guarantee state isolation.

Chapter XXX

Lessons learned

Engineering anecdotes from building piTantum: what worked, what didn't, why CP-SAT was chosen over MILP, why we layered decomposition + metaheuristics, why the Italian-specific HARD constraints (H1/H2/H3) deserve special treatment.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/lessons_learned.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

Chapter XXXI

Glossary (narrative form)

Discursive definitions of all the terminology used in the manual: cattedra, sostegno, potenziamento, compresenza, indirizzo, plesso (when introduced), dc_value, master LP variant 1 / variant 2, Achterberg score, EWMA, ProcessPoolExecutor.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/glossario_discorsivo.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Chapter XXXII

Reference

Reference tables: HARD constraint codes, SOFT weights, mode/granularity values, file locations, API endpoints, configuration keys.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/reference.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

Analytical index